UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática

Actualización del núcleo tecnológico de EDDIE

Nicolás Alonso Muñoz Reyes

Profesor guía: Roberto González Ibáñez

Tesis de grado presentada en conformidad a los requisitos para obtener el título de Ingeniero Civil en Informática

Santiago – Chile
2026

RESUMEN

EDDIE es una plataforma de investigación del laboratorio InTeractiOn de la Universidad de Santiago de Chile que habilita la lectura aumentada no obstructiva sobre documentos impresos: un proyector y dos cámaras sobre el escritorio permiten resaltar una palabra en el papel y obtener su definición, traducción o contenido relacionado, sincronizar anotaciones entre el libro y su versión PDF y registrar la mirada del lector con un eye tracker. Tras seis memorias de título sucesivas, el sistema llegó a unas 45 000 líneas de C# en 26 proyectos, con tres mecanismos distintos de carga de módulos por reflexión y 67 errores documentados por Ibaceta Jaña (2023), lo que impedía usarlo de forma confiable como instrumento de medición.

Este trabajo reconstruye el núcleo de EDDIE en Python bajo una arquitectura de plugins con un contrato único (`PluginBase`), un orquestador central (`OrchestratorCore`) que ejecuta un ciclo de inicialización, ejecución y liberación con aislamiento de fallos, y una capa de abstracción de hardware (`DeviceFactory`) que conmuta cada dispositivo entre su implementación real y una simulada. Siguiendo el ciclo PDCA con prototipado rápido (RAD) y Kanban, se evaluó primero la factibilidad de migrar 19 funciones críticas mediante prototipos desechables y luego se integraron, en siete iteraciones, los módulos de consistencia físico-digital, reconocimiento de texto, interacción gestual, rastreo ocular, búsqueda (respondida por un modelo de lenguaje en lugar de las APIs vencidas) y voz, con una interfaz rediseñada a partir de los hallazgos de usabilidad de la evaluación previa.

La versión final aprueba 107 pruebas automatizadas con una cobertura de líneas de 81 % y una latencia del ciclo completo del orquestador de 1,48 ms en el percentil 95, bajo el umbral de 8,3 ms derivado de la frecuencia de los eye trackers. La verificación funcional con MECIVA PS y las pruebas de aceptación con el hardware del laboratorio completan la evaluación.

Palabras clave: lectura aumentada; arquitectura de plugins; software científico; deuda técnica; rastreo ocular; Python.

ABSTRACT

EDDIE is a research platform from the InTeractiOn laboratory at Universidad de Santiago de Chile that enables non-obstructive augmented reading on printed documents: a projector and two cameras above the desk let readers highlight a word on paper and obtain its definition, translation or related content, synchronize annotations between the book and its PDF, and record their gaze with an eye tracker. After six successive undergraduate theses, the system had grown to about 45,000 lines of C# in 26 projects, with three different reflection-based module loading mechanisms and 67 documented errors (Ibaceta Jaña, 2023), which prevented its reliable use as a measurement instrument.

This work rebuilds the EDDIE core in Python on a plugin architecture with a single contract (`PluginBase`), a central orchestrator that runs an initialize–execute–release cycle with fault isolation, and a hardware abstraction layer that switches each device between its real and simulated implementation. Following PDCA with rapid application development and Kanban, the feasibility of migrating 19 critical functions was first assessed with throwaway prototypes; the consistency, text recognition, gesture, eye tracking, search (answered by a language model instead of the expired web APIs) and voice modules were then integrated in seven iterations, with a user interface redesigned from previous usability findings.

The final version passes 107 automated tests with 81 % line coverage and a 95th-percentile orchestrator cycle latency of 1,48 ms, below the 8.3 ms threshold derived from the eye trackers' sampling rate. Functional verification with MECIVA PS and acceptance tests with the laboratory hardware complete the evaluation.

Keywords: augmented reading; plugin architecture; research software; technical debt; eye tracking; Python.

TABLA DE CONTENIDO

Resumen I

Abstract II

1. Introducción 1

1.1. Antecedentes y motivación . . . 1
1.2. Descripción del problema . . . 2
1.3. Solución propuesta . . . 2
1.4. Objetivos y alcance del proyecto . . . 3
1.4.1. Objetivo general . . . 3
1.4.2. Objetivos específicos . . . 4
1.4.3. Alcances y limitaciones . . . 4
1.5. Metodología y herramientas utilizadas . . . 5
1.5.1. Metodología . . . 5
1.5.2. Herramientas de desarrollo . . . 6
1.5.3. Ambiente de desarrollo . . . 7
1.6. Organización del documento . . . 7

2. Marco teórico 8

2.1. Marco conceptual . . . 8
2.1.1. Lectura aumentada y realidad aumentada espacial . . . 8
2.1.2. Percepción del documento: binarización y OCR . . . 8
2.1.3. Rastreo ocular e interacción por la mirada . . . 9
2.1.4. Software científico y deuda técnica . . . 9
2.1.5. Arquitectura de plugins y orquestación . . . 10
2.1.6. MECIVA PS . . . 10
2.2. Estado del arte . . . 10
2.2.1. Antecedentes del proyecto (evolución histórica) . . . 10
2.2.1.1. Catálogo de funcionalidades y levantamiento de la usabilidad previa . . . 12
2.2.2. Sistemas de lectura aumentada similares . . . 13
2.2.3. El problema de la modernización en software de investigación . . . 15

2.2.4. Multimodal Observer como referente arquitectural 15

3. Análisis **17**

3.1. Diagnóstico 17
3.1.1. Reclasificación de los errores de EDDIE-2023 17
3.1.2. Arquitectura en C# 18
3.2. Captura de requerimientos 19
3.2.1. Fuentes y etapas del proceso 19
3.2.2. Requerimientos funcionales 20
3.2.3. Requerimientos no funcionales 23
3.3. Factibilidad 24
3.3.1. Factibilidad técnica 24
3.3.2. Factibilidad económica 24
3.3.3. Factibilidad operativa 24
3.4. Prototipado 24
3.4.1. Inventario de funciones 25
3.4.2. Síntesis de la selección y método de medición 28
3.4.3. Prototipo F1: Enumeración e inicialización de cámaras 29
3.4.4. Prototipo F2: Preprocesamiento y binarización 30
3.4.5. Prototipo F3: Reconocimiento óptico de caracteres 31
3.4.6. Prototipo F4: Detección automática de páginas 33
3.4.7. Prototipo F5: Seguimiento de lápiz de color 33
3.4.8. Prototipo F6: Segmentación de mano y apuntado 34
3.4.9. Prototipo F7: Mapeo de coordenadas 35
3.4.10. Prototipo F9: Mouse como alternativa 36
3.4.11. Prototipo F10: Comunicación con el eye tracker 37
3.4.12. Prototipo F11: Carga dinámica de plugins 38
3.4.13. Prototipo F12: Clic por permanencia de la mirada 39
3.4.14. Prototipo F13: Retícula de mirada proyectada 41
3.4.15. Prototipo F14: Búsqueda de coordenadas QuadPoints 42
3.4.16. Prototipo F15: Sincronización de resaltado 43
3.4.17. Prototipo F16: Sincronización de notas adhesivas 44
3.4.18. Prototipo F17: Sincronización de figuras geométricas 45
3.4.19. Prototipo F24: Comandos de voz 46
3.4.20. Prototipo F25: Síntesis de voz 47
3.4.21. Prototipo F27: Banda de resaltado proyectada 48

3.4.22. Síntesis de los prototipos 48

4. Diseño y aspectos de implementación 50
4.1. Arquitectura general 50
4.2. Contratos de plugin 51
4.3. Núcleo orquestador 52
4.3.1. Ciclo de vida 52
4.3.2. Configuración declarativa 53
4.3.3. Capa de abstracción de hardware 54
4.3.4. Integración con la interfaz 54
4.4. Módulos integrados 54
4.5. Diseño de la interfaz 55
4.6. Iteraciones de desarrollo 58

5. Evaluación 59
5.1. Plan de evaluación 59
5.2. Pruebas de software 60
5.2.1. Pruebas automatizadas y cobertura 60
5.2.2. Latencia del orquestador 61
5.2.3. Aislamiento de fallos 62
5.2.4. Matriz de trazabilidad 62
5.2.5. Hallazgos de la verificación 63
5.3. Verificación y validación con MECIVA PS 64
5.3.1. Protocolo 64
5.3.2. Resultados 65
5.4. Pruebas de aceptación 65
5.5. Síntesis de la evaluación 66

6. Conclusiones 68
6.1. Cumplimiento de los objetivos 68
6.2. Respuesta a la pregunta de investigación 69
6.3. Limitaciones 69
6.4. Trabajo futuro 70

Glosario 71

A. Errores de EDDIE-2023 y plantilla de trazabilidad 76

B. Plan de trabajo **78**

A. Fichas de los avances **79**

B. Mediciones de los prototipos **85**

C. Comparación de la interfaz **87**

D. Manual de instalación y uso **89**

D.1. Instalación 89

D.2. Ejecución 89

D.3. Uso 89

ÍNDICE DE TABLAS

2.1. Síntesis de la evolución histórica del sistema EDDIE a través de seis memorias de título sucesivas. 12
2.2. Comparación de sistemas de lectura aumentada. 15

3.1. Reclasificación por causa de los 67 errores documentados por Ibaceta Jaña (2023). 18
3.2. Requerimientos funcionales de EDDIE por módulo. 20
3.3. Requerimientos no funcionales y su criterio de aceptación. 23
3.4. Funciones del subsistema de procesamiento de imágenes y visión computacional. 25
3.5. Funciones del subsistema de reconocimiento gestual. 26
3.6. Funciones del subsistema de rastreo ocular. 26
3.7. Funciones del subsistema de consistencia físico-digital. 27
3.8. Funciones del subsistema de búsqueda. 27
3.9. Funciones del subsistema de voz. 28
3.10. Funciones del subsistema de presentación, proyección y registro. 28
3.11. Ficha de validación del prototipo: Enumeración e inicialización de cámaras. 29
3.12. Ficha de validación del prototipo: Preprocesamiento y binarización. . . . 30
3.13. Ficha de validación del prototipo: Reconocimiento óptico de caracteres. . 32
3.14. Ficha de validación del prototipo: Detección automática de páginas. . . . 33
3.15. Ficha de validación del prototipo: Seguimiento de lápiz de color. 34
3.16. Ficha de validación del prototipo: Segmentación de mano y apuntado. . . 35
3.17. Ficha de validación del prototipo: Mapeo de coordenadas (homografía). . 36
3.18. Ficha de validación del prototipo: Mouse como alternativa. 37
3.19. Ficha de validación del prototipo: Comunicación con el eye tracker. 38
3.20. Ficha de validación del prototipo: Carga dinámica de plugins. 39
3.21. Ficha de validación del prototipo: Clic por permanencia de la mirada. . . . 40
3.22. Barrido de los parámetros del clic por permanencia ($\sigma = 10$ px). 41
3.23. Ficha de validación del prototipo: Retícula de mirada proyectada. 42
3.24. Ficha de validación del prototipo: Búsqueda de coordenadas QuadPoints. 43
3.25. Ficha de validación del prototipo: Sincronización de resaltado. 44

3.26. Ficha de validación del prototipo: Sincronización de notas adhesivas. . . 45
3.27. Ficha de validación del prototipo: Sincronización de figuras geométricas. 46
3.28. Ficha de validación del prototipo: Comandos de voz. 47
3.29. Ficha de validación del prototipo: Síntesis de voz. 47
3.30. Ficha de validación del prototipo: Banda de resaltado proyectada. 48

4.1. Contratos de plugin y sus implementaciones. 52
4.2. Avances del desarrollo y resultados de sus pruebas. 58

5.1. Métricas de evaluación, criterio de aceptación y estado. 60
5.2. Latencia por plugin en 500 ciclos del orquestador (ms). 61
5.3. Matriz de trazabilidad (parcial) de errores de EDDIE-2023. 63
5.4. Hallazgos de la verificación y su corrección. 64
5.5. Escala de evaluación de las pruebas de aceptación. 66
5.6. Pruebas de aceptación planificadas. 66

A.1. Extracto de errores encontrados en EDDIE-2023. 76
A.2. Plantilla de la matriz de trazabilidad. 77

B.1. Plan de trabajo. 78

A.1. Primer prototipo - Núcleo orquestador y contratos de plugins. 79
A.2. Segundo prototipo - Capa de abstracción de hardware y módulo de consistencia. 80
A.3. Tercer prototipo - Reconocimiento de texto e interfaz de configuración. . . 81
A.4. Cuarto prototipo - Interacción gestual. 82
A.5. Quinto prototipo - Rastreo ocular. 82
A.6. Sexto prototipo - Pantalla de proyección, búsqueda con LLM y voz. 83
A.7. Séptimo prototipo - Versión final verificada. 83
A.8. Hallazgos de la verificación. 84

B.1. Resumen de las mediciones de los prototipos. 85

ÍNDICE DE ILUSTRACIONES

2.1. Primer prototipo de DigitalDesk. 14
2.2. Gesto de señalamiento en EnhancedDesk. 14
2.3. Descripción general de AR-PaperSync. 14
2.4. Enlight: lectura aumentada. 14
2.5. Multimodal Observer: vista principal. 16

3.1. Arquitectura de EDDIE-2023: tres mecanismos de carga independientes. 19

4.1. Arquitectura en capas de EDDIE en Python. 51
4.2. Pantalla de inicio con los pasos de preparación y el estado de cada módulo. 56
4.3. Configuración del reconocimiento de texto (izquierda) y de la interacción gestual con lápiz (derecha). 57
4.4. Configuración del documento PDF (izquierda) y del rastreo ocular (derecha). 57
4.5. Pantalla proyectada: controles en la franja superior (izquierda) y resultado de una búsqueda (derecha). 57

C.1. Pantalla de inicio: EDDIE-2023 (izquierda) y versión en Python (derecha). 87
C.2. Configuración de gestos: EDDIE-2023 (izquierda) y versión en Python (derecha). 87
C.3. Pantalla proyectada: EDDIE-2023 (izquierda) y versión en Python (derecha). 88

ÍNDICE DE ALGORITMOS

1. Ciclo de vida del orquestador. 53

CAPÍTULO 1. INTRODUCCIÓN

1.1 ANTECEDENTES Y MOTIVACIÓN

La lectura constituye un proceso cognitivo fundamental para el aprendizaje y la construcción de conocimiento, el cual ha experimentado una transformación sustancial con la digitalización de la información. En el ámbito de la Interacción Humano-Información (HII) resulta de especial interés analizar cómo las personas interactúan con distintos soportes de lectura y de qué manera estas modalidades impactan en factores como la comprensión lectora, la retención de ideas y la carga cognitiva. A pesar de la masificación de los dispositivos electrónicos, la evidencia empírica muestra que el material impreso conserva vigencia y sigue siendo preferido para tareas de lectura profunda (Saleh y Mashhur, 2015; Singer y Alexander, 2016, como se citaron en Santibáñez Espinoza, 2020).

Para avanzar en el estudio sistemático de estos fenómenos, la comunidad científica requiere herramientas y metodologías experimentales rigurosas. La investigación en este campo depende de entornos que integren componentes de hardware, software y diseños experimentales capaces de capturar y evaluar, de manera precisa y no intrusiva, las dinámicas de interacción del lector durante el proceso de lectura. Dentro de las aproximaciones tecnológicas en esta área, una de las líneas menos exploradas es la lectura aumentada aplicada sobre papel físico. En este contexto, la lectura aumentada no se refiere a la superposición de animaciones o elementos tridimensionales lúdicos, sino al enriquecimiento del material impreso con prestaciones funcionales equivalentes a las del formato digital: búsqueda en línea de términos del texto, traducción instantánea, consulta de definiciones en contexto, despliegue de contenido complementario y sincronización bidireccional de anotaciones entre el documento físico y su versión digital.

Las herramientas capaces de soportar este tipo de experiencias integradas son escasas tanto en la literatura como en la práctica experimental. En el Departamento de Ingeniería Informática de la Universidad de Santiago de Chile (USACH), bajo la línea de trabajo del grupo InTeractiOn, nació la plataforma EDDIE (*augmEnteD reaDIng dEsk*), concebida como un instrumento de investigación para habilitar experiencias de lectura aumentada no obstructiva sobre documentos impresos (Carrasco Madriaga,

2017). A lo largo de los años, EDDIE ha experimentado múltiples fases de desarrollo e integración incremental a través de aportes de distintos memoristas, quienes incorporaron capacidades de procesamiento visual, búsqueda, consistencia y seguimiento ocular (Ibaceta Jaña, 2023).

Sin embargo, debido al tiempo transcurrido desde su concepción original y a la acumulación de evoluciones heterogéneas desarrolladas en diferentes etapas, la herramienta presenta limitaciones y desactualizaciones operativas que impiden utilizarla de manera plena, estable y confiable en estudios experimentales contemporáneos sobre comprensión y comportamiento lector, limitando su potencial como plataforma de medición científica.

1.2 DESCRIPCIÓN DEL PROBLEMA

El problema central del proyecto EDDIE no radica en el volumen total de errores documentados, sino en un riesgo arquitectónico específico sobre una base de aproximadamente 45 000 líneas de código distribuidas en 26 proyectos. De los 67 problemas registrados por Ibaceta Jaña (2023), su reclasificación por causa (Tabla 3.1) muestra que 16 (23 %) corresponden a la infraestructura técnica de carga y despliegue, mientras que el resto se distribuye entre errores de práctica de código, hardware y sensores, servicios externos, interfaz e idioma, infraestructura de laboratorio y documentación. Aunque los cuatro errores de carga por reflexión representan solo una fracción de esta infraestructura técnica, tienen un impacto desproporcionado: cada uno impide el arranque completo de un módulo, y su causa raíz constituye un riesgo estructural recurrente, ya que cualquier desarrollador que integre un nuevo plugin puede reproducir el mismo tipo de falla al no existir una única interfaz validada que lo prevenga.

Ante este escenario, se plantea la pregunta: ¿en qué medida la reconstrucción del núcleo de EDDIE bajo una arquitectura de plugins con contrato único permite reducir el número de errores estructurales documentados y habilitar su uso como instrumento de medición científica confiable?

1.3 SOLUCIÓN PROPUESTA

Se propone reconstruir el núcleo tecnológico de EDDIE en Python bajo una arquitectura de plugins con un contrato único, siguiendo el antecedente validado de Multimodal Observer (MO) en el mismo laboratorio (Figueroa Contreras, 2025). La solución reemplaza los tres mecanismos heterogéneos de carga por reflexión del sistema en

C# (`Assembly.LoadFrom`, `AppDomain` y `Assembly.LoadFile`) por un único mecanismo, y se compone de:

- Un **orquestador central** (`OrchestratorCore`) que ejecuta en un único proceso un ciclo estandarizado de tres fases (inicialización, ejecución y liberación) y aísla en un manejo de excepciones centralizado la falla de cualquier plugin, de modo que queda registrada sin detener el sistema.
- Un **cargador de plugins** (`PluginLoader`) que instancia, a partir de un único archivo de configuración (`config/plugins.json`), las clases declaradas mediante `importlib`.
- Un **contrato base** (`PluginBase`) y contratos por dominio: `ICameraSensor`, `IImageProcessor`, `IGesturePlugin`, `IEyeTracker`, `IConsistencyProvider`, `IWebSearchProvider` e `ISpeechProvider`.
- Una **capa de abstracción de hardware** (`DeviceFactory`) que conmuta cada dispositivo entre su implementación real y una simulada mediante una variable de entorno, lo que permite validar el sistema completo sin el laboratorio antes de una sesión real.
- La integración de los módulos existentes como plugins: consistencia bidireccional papel–PDF, reconocimiento del texto resaltado, interacción gestual, rastreo ocular, búsqueda y voz, junto con la interfaz de configuración y la pantalla proyectada.

Respecto de la propuesta original, en la que `IWebSearchProvider` quedaba solo como interfaz, los cinco servicios de búsqueda (enciclopedia, definiciones, traducción, video e imágenes) se implementaron sobre un modelo de lenguaje (Google Gemini 2.5 Flash), ya que las APIs que usaba el sistema legado (Wikipedia, Google, Bing, YouTube y Cloud Vision) dejaron de estar disponibles en sus condiciones originales.

1.4 OBJETIVOS Y ALCANCE DEL PROYECTO

1.4.1 Objetivo general

Actualizar el núcleo tecnológico del proyecto EDDIE mediante el rediseño y reimplementación en Python e integración de los módulos de investigación existentes, para reducir la deuda técnica y asegurar la compatibilidad con sensores físicos del proyecto en un entorno de lectura aumentada no obstructiva.

1.4.2 Objetivos específicos

1. Diagnosticar la arquitectura actual de EDDIE en C# a partir de la reclasificación de errores de Ibaceta Jaña (2023), y evaluar mediante prototipos desechables en Python la factibilidad técnica de migrar los módulos existentes a las tecnologías propuestas.

2. Diseñar el núcleo orquestador de EDDIE, definiendo los contratos de interfaz que permitan la integración de los módulos previos como plugins desacoplados.

3. Implementar el núcleo orquestador en Python, junto con la capa de abstracción de sensores, integrando los módulos de consistencia bidireccional, reconocimiento de texto, interacción gestual y rastreo ocular bajo la nueva arquitectura.

4. Validar el sistema integrado mediante pruebas de software, verificación funcional y pruebas de aceptación en el ambiente operativo real del laboratorio InTeractiOn.

1.4.3 Alcances y limitaciones

La solución cubre la reconstrucción completa del núcleo orquestador, los contratos de interfaz de plugin y la capa de abstracción de hardware con conmutación real/simulado. La validación con hardware real se limita a los cuatro módulos del MVP (`IConsistencyProvider`, `IEyeTracker` sobre GazePoint y TheEyeTribe, `IImageProcessor` e `IGesturePlugin`) en el ambiente del laboratorio InTeractiOn. La búsqueda con modelo de lenguaje y la voz se integraron como extensión del alcance, pero su validación se limita a pruebas de software con un servidor de lenguaje local de prueba, ya que dependen de un servicio externo y de una clave personal.

Quedan fuera del alcance el dashboard web, el visor de proyección en navegador, la interfaz de línea de comandos de validación y su canal de telemetría WebSocket, el reconocimiento automático del libro físico y el catálogo de lectura, la eliminación de accesorios mediadores (mouse, lápices de color) y la recalibración de visión computacional más allá de la mínima necesaria para el MVP. La migración de Emgu CV a OpenCV no es una traducción directa, ya que ambas bibliotecas pueden producir resultados numéricos distintos ante los mismos parámetros, por lo que los algoritmos de reconocimiento deben recalibrarse y validarse en el nuevo entorno.

Al ejecutarse todos los plugins como objetos en memoria dentro del proceso del orquestador se elimina el costo de comunicación entre procesos, pero el aislamiento de fallos depende del manejo de excepciones centralizado (Capítulo 4). La proyección

se dibuja en el mismo proceso mediante una ventana nativa (Qt). Se eliminan las dependencias nativas de 32 bits, lo que mejora la portabilidad potencial del software; sin embargo, el desarrollo y la validación se realizan exclusivamente en Windows 11, el sistema depende de SDK de fabricante específicos de esa plataforma y está atado a un montaje físico particular, por lo que la verificación en Linux o macOS queda fuera de alcance.

1.5 METODOLOGÍA Y HERRAMIENTAS UTILIZADAS

1.5.1 Metodología

Para la modernización del núcleo de EDDIE se emplea el ciclo PDCA (*Plan-Do-Check-Act*), complementado con Kanban para el seguimiento de tareas y *Rapid Application Development* (RAD) para el desarrollo iterativo mediante prototipos. Esta combinación replica el enfoque validado por Figueroa Contreras (2025) en la reimplementación de MO, dada la similitud estructural entre ambos proyectos. El ciclo PDCA estructura el trabajo en cuatro etapas, correspondientes a los objetivos específicos:

- **Planificación (Plan, OE1):** diagnóstico del estado actual de EDDIE e identificación de los nudos críticos de incompatibilidad mediante prototipos desechables (avance P-00).
- **Ejecución (Do, OE2 y OE3):** diseño e implementación del núcleo orquestador, la capa de abstracción de sensores y la integración de los módulos como plugins mediante iteraciones sucesivas de prototipado (avances P-01 a P-06).
- **Verificación (Check, OE4):** pruebas de software, verificación funcional y pruebas de aceptación descritas en el Capítulo 5.
- **Actuación (Act, OE4):** incorporación de los ajustes derivados de dichas pruebas y cierre de la documentación técnica (avance P-07).

Un aspecto central de RAD es el uso de prototipos desechables (*throwaway prototyping*) durante las primeras semanas, en lugar de incorporar directamente cada funcionalidad migrada al código definitivo. Cada componente crítico se valida primero mediante un prototipo aislado y de bajo costo, cuyo único propósito es confirmar la factibilidad técnica de la migración a Python antes de invertir tiempo en su integración formal al orquestador (Figueroa Contreras, 2025). Kanban se utiliza para visualizar el

flujo de tareas de cada módulo y fase del ciclo PDCA a lo largo de las 17 semanas del proyecto (Tabla B.1). Cada iteración se registra como un avance con código completo, pruebas y evidencias reproducibles (Apéndice A).

1.5.2 Herramientas de desarrollo

Las herramientas empleadas durante el desarrollo del proyecto son las siguientes:

- **Python 3.12**, lenguaje principal del núcleo orquestador, la capa de abstracción de sensores y los plugins.
- **OpenCV** (Bradski, 2000), para las funciones de visión computacional que en EDDIE-2023 dependían de Emgu CV.
- **Tesseract OCR** (Smith, 2007) mediante `pytesseract`, para el reconocimiento del texto resaltado.
- **PyMuPDF**, para la consistencia con el documento PDF, en reemplazo de iTextSharp.
- **PySide6** (Qt 6), para la interfaz de configuración y la pantalla proyectada, en reemplazo de WinForms.
- **Google Gemini 2.5 Flash**, modelo de lenguaje que responde los servicios de búsqueda.
- **pytest** y **pytest-cov**, para las pruebas unitarias, de integración y de interfaz y la medición de cobertura.
- **SonarQube**, para el análisis estático de calidad del código durante la etapa de verificación.
- **Git y GitHub**, para el control de versiones; **Visual Studio Code**, como entorno de desarrollo.
- **EDDIE-2023** (versión en C#), como referencia funcional y línea base de comparación.
- **Overleaf** (LaTeX) para la redacción del informe y **Notion** para el tablero Kanban y el registro de avances.

1.5.3 Ambiente de desarrollo

El desarrollo se realiza en dos ambientes complementarios. El primero corresponde a un equipo personal utilizado para la implementación y las pruebas unitarias y de integración del núcleo y los plugins, con las siguientes características:

- Procesador AMD Ryzen 5 7600, 6 núcleos, 3,80 GHz.
- Memoria RAM de 32 GB (5200 MT/s).
- Tarjeta gráfica NVIDIA GeForce RTX 5060 (8 GB VRAM) y GPU integrada AMD Radeon.
- Almacenamiento SSD de 932 GB.
- Sistema operativo Windows 11 Pro (versión 25H2).

En este ambiente se realiza la mayor parte del ciclo de desarrollo iterativo, con los dispositivos simulados de la capa de abstracción de hardware. El segundo ambiente corresponde a la sala de experimentación del Departamento de Ingeniería Informática de la USACH, donde se encuentra el despliegue físico de EDDIE: el computador designado para su operación, dos cámaras digitales para el reconocimiento de texto y el reconocimiento gestual, un proyector que actúa como pantalla sobre el escritorio y los eye trackers GazePoint y TheEyeTribe. Este ambiente se utiliza en la etapa de verificación para las pruebas de aceptación y la validación con el hardware físico real.

1.6 ORGANIZACIÓN DEL DOCUMENTO

El Capítulo 2 presenta el marco conceptual y el estado del arte: la evolución histórica de EDDIE, los sistemas de lectura aumentada comparables, el problema de la modernización del software científico y el antecedente de Multimodal Observer. El Capítulo 3 contiene el diagnóstico de la arquitectura en C#, la captura de requerimientos, el análisis de factibilidad técnica y los prototipos desechables con sus mediciones. El Capítulo 4 describe la arquitectura del núcleo, los contratos de plugin, la capa de abstracción de hardware, los módulos integrados, la interfaz y las iteraciones de desarrollo. El Capítulo 5 reporta las pruebas de software, los hallazgos de la verificación y el protocolo y resultados de la verificación funcional y las pruebas de aceptación. El Capítulo 6 presenta las conclusiones, las limitaciones y el trabajo futuro. Los anexos reúnen la reclasificación de errores y el plan de trabajo; los apéndices, las tablas de los avances, las mediciones detalladas y el manual de instalación y uso.

CAPÍTULO 2. MARCO TEÓRICO

2.1 MARCO CONCEPTUAL

Esta sección define los conceptos que se usan en el resto del documento. Se agrupan en tres bloques: la lectura aumentada y sus técnicas de percepción, la interacción mediante la mirada, y los conceptos de ingeniería de software con los que se aborda la reconstrucción del sistema.

2.1.1 Lectura aumentada y realidad aumentada espacial

Se entiende por **lectura aumentada** el enriquecimiento de un documento impreso con prestaciones propias del formato digital (búsqueda, definición, traducción, anotación sincronizada) sin reemplazar el papel como soporte de lectura. Cuando la información digital se dibuja directamente sobre los objetos físicos mediante un proyector, en lugar de mostrarse en una pantalla o un visor que el usuario sostiene o lleva puesto, se habla de **realidad aumentada espacial** (Bimber and Raskar, 2005). Esta modalidad es **no obstructiva**: el lector no necesita sostener un dispositivo ni llevar un casco, lo que la hace adecuada para estudiar la lectura sin alterar la conducta que se quiere medir.

Un sistema de este tipo necesita relacionar tres sistemas de coordenadas: el de la cámara que observa el escritorio, el de la imagen del proyector y el del documento PDF. La relación entre dos planos observados en perspectiva se modela con una **homografía**, una transformación proyectiva $H \in \mathbb{R}^{3\times 3}$ que lleva un punto homogéneo $p = (x, y, 1)^\top$ de un plano a otro, $p' \sim Hp$, y que puede estimarse con al menos cuatro pares de puntos correspondientes (Hartley and Zisserman, 2004). En EDDIE la homografía se obtiene de una calibración de cuatro puntos y permite convertir la posición de un lápiz o de un dedo en la cámara en una posición sobre la proyección.

2.1.2 Percepción del documento: binarización y OCR

La **binarización** separa los píxeles de tinta de los del fondo. El método de Otsu (1979) elige el umbral t^* que maximiza la varianza entre las dos clases del histograma de grises,

$$t^* = \arg\max_{t} \; \omega_0(t)\,\omega_1(t)\,\big[\mu_0(t) - \mu_1(t)\big]^2, \tag{2.1}$$

donde ω_i y μ_i son la proporción y la media de cada clase. Es un umbral *global*: un único valor para toda la imagen, por lo que se degrada cuando la iluminación no es uniforme, como ocurre con la luz de un proyector sobre papel. Los umbrales *adaptativos* calculan un valor por vecindad y toleran mejor esos gradientes.

El **reconocimiento óptico de caracteres** (OCR) convierte la imagen de un texto en una cadena de caracteres. EDDIE usa el motor Tesseract (Smith, 2007). Su calidad se mide con la tasa de error por carácter (CER) y por palabra (WER), que cuentan las sustituciones (S), supresiones (D) e inserciones (I) necesarias para transformar la transcripción en el texto de referencia de largo N, es decir, su distancia de edición de Levenshtein (1966):

$$\mathrm{CER} = \frac{S_c + D_c + I_c}{N_c} \times 100\,\%, \qquad \mathrm{WER} = \frac{S_w + D_w + I_w}{N_w} \times 100\,\%. \tag{2.2}$$

2.1.3 Rastreo ocular e interacción por la mirada

Un **eye tracker** estima, a una frecuencia de muestreo fija (60 Hz en GazePoint y TheEyeTribe), el punto de la escena al que mira una persona. La señal se compone de **fijaciones**, períodos en que la mirada permanece estable sobre una región y en los que se procesa la información, y **sacadas**, desplazamientos rápidos entre fijaciones. Un método habitual para identificar fijaciones es el de dispersión (I-DT), que agrupa muestras consecutivas mientras su dispersión no supere un umbral durante un tiempo mínimo (Salvucci and Goldberg, 2000).

Cuando la mirada se usa además para *actuar* (seleccionar un botón), el sistema debe distinguir entre mirar para leer y mirar para activar. El **clic por permanencia** (*dwell time*) dispara la acción cuando la mirada permanece dentro de un radio R durante un tiempo T. Si T es corto, cualquier fijación de lectura activa controles involuntariamente, problema conocido como **toque de Midas** (Jacob, 1990); si es largo, la interacción se vuelve lenta. La elección de (T, R) es, por lo tanto, un compromiso que debe medirse.

2.1.4 Software científico y deuda técnica

En este trabajo se entiende por **software científico** aquel desarrollado *ad hoc* para soportar una tarea de investigación específica, como la captura, el procesamiento o el análisis de datos experimentales, en contraposición al software de propósito general que un laboratorio utiliza como herramienta auxiliar. Bajo esta definición, EDDIE

y Multimodal Observer son software científico, mientras que el sistema operativo o el editor de código empleados en su desarrollo no lo son.

La **deuda técnica** es el costo futuro que generan las decisiones de diseño o implementación que resuelven un problema inmediato a expensas de la mantenibilidad. En el software científico aparece con frecuencia porque cada investigador optimiza para su propio trabajo y no para el sistema común (Ernst et al., 2026).

2.1.5 Arquitectura de plugins y orquestación

Una **arquitectura de plugins** separa un núcleo estable de módulos intercambiables que implementan un **contrato de interfaz** común, de modo que se puede agregar o reemplazar un módulo sin modificar el núcleo (Bustio Martínez et al., 2013). El **orquestador** es el componente del núcleo que conoce a todos los plugins, controla su ciclo de vida (inicialización, ejecución y liberación) y coordina el flujo de datos entre ellos, a diferencia de un esquema de coreografía en que los módulos se coordinan entre sí sin un componente central (Megargel et al., 2021). Una **capa de abstracción de hardware** (HAL) oculta detrás de una interfaz los detalles de cada dispositivo físico, lo que permite sustituirlo por una implementación simulada para pruebas.

2.1.6 MECIVA PS

MECIVA PS es la metodología de evaluación y validación de software científico aplicada por Ibaceta Jaña (2023) a EDDIE. Clasifica los requisitos en críticos y secundarios, ejecuta verificaciones funcionales y no funcionales, registra los errores encontrados con su estado (sin falla, solución parcial, sin solución) y evalúa la validez y la confiabilidad de las mediciones del sistema mediante ejecuciones repetidas. En este trabajo se usa como protocolo de verificación del sistema reconstruido, de modo que sus resultados sean comparables con la línea base de 2023 (Capítulo 5).

2.2 ESTADO DEL ARTE

2.2.1 Antecedentes del proyecto (evolución histórica)

Para comprender los fundamentos de la presente propuesta de modernización es necesario reconstruir la trayectoria histórica de EDDIE (*augmEnteD reaDIng dEsk*), desarrollado de forma incremental en el Departamento de Ingeniería Informática de la Universidad de Santiago de Chile bajo el alero del laboratorio InTeractiOn. El proyecto

es un caso representativo de software científico que evolucionó a través de seis memorias de título sucesivas, cada una de las cuales incorporó un módulo funcional sin que existiera una revisión sistemática de la arquitectura común.

El concepto original fue propuesto por Carrasco Madriaga (2017), quien implementó un entorno de realidad aumentada orientado a enriquecer la lectura en papel. Contreras Velásquez (2019) evolucionó esta propuesta hacia una configuración no obstructiva de escritorio con cámara y proyector fijos. A partir de esta base, el proyecto se expandió con tres capacidades de soporte al lector: Montanares Mundaca (2019) integró un módulo de búsqueda web contextual activado por voz y gestos; Pezoa Riquelme (2020) desarrolló un mecanismo de consistencia bidireccional entre el documento físico y su versión PDF; y Estay Vera (2020) incorporó dispositivos de rastreo ocular para el análisis del comportamiento atencional. Posteriormente, Santibáñez Espinoza (2020) rediseñó la interfaz gráfica bajo un enfoque centrado en el usuario y, finalmente, Ibaceta Jaña (2023) aplicó la metodología MECIVA PS para diagnosticar el estado del software, identificando 67 errores (Tabla 2.1).

Este proceso incremental produjo un sistema de aproximadamente 45 000 líneas de código distribuidas en 26 proyectos `.csproj`, en el que la falta de una arquitectura común llevó a la coexistencia de tres mecanismos heterogéneos de carga por reflexión: `Assembly.LoadFrom`, un `AppDomain` subordinado y `Assembly.LoadFile`. Los errores de carga y despliegue suman 16 de los 67 documentados (Tabla 3.1).

Tabla 2.1: Síntesis de la evolución histórica del sistema EDDIE a través de seis memorias de título sucesivas.

Trabajo	**Aporte al proyecto EDDIE**	**Limitaciones / deuda técnica generada**
Carrasco Madriaga (2017)	Prototipo inicial de lectura aumentada con reconocimiento del texto resaltado.	Dependencia de mediadores manuales.
Contreras Velásquez (2019)	Configuración no obstructiva de escritorio (cámara y proyector) y reconocimiento gestual.	Acoplamiento directo con la interfaz WinForms en C#.
Montanares Mundaca (2019)	Módulo de búsqueda web contextual (enciclopedia, definiciones, traducción, video, imágenes) y voz.	Carga mediante reflexión con `Assembly.LoadFile`; APIs con credenciales de estudiante.
Pezoa Riquelme (2020)	Consistencia bidireccional físico-digital (PDF).	Rutas fijas y carpeta de plugins de consistencia que debía crearse a mano.
Estay Vera (2020)	Integración de rastreo ocular (GazePoint, TheEyeTribe, GazeCloud).	Carga aislada en un `AppDomain` subordinado.
Santibáñez Espinoza (2020)	Rediseño centrado en el usuario de la interfaz gráfica.	Mantenimiento de la capa monolítica subyacente.
Ibaceta Jaña (2023)	Diagnóstico de errores mediante la metodología MECIVA PS.	Registro de 67 errores; varios sin solución en hardware real.

Fuente: Elaboración propia a partir de los trabajos citados.

2.2.1.1 Catálogo de funcionalidades y levantamiento de la usabilidad previa

En su estado actual, EDDIE ofrece cinco categorías funcionales heredadas de las memorias descritas: procesamiento de imágenes y reconocimiento gestual (Contreras Velásquez, 2019), búsqueda web contextual (Montanares Mundaca, 2019), consistencia bidireccional entre el documento físico y su versión digital (Pezoa Riquelme, 2020), rastreo ocular (Estay Vera, 2020) y una capa de presentación en WinForms resultante del rediseño de Santibáñez Espinoza (2020). Cada funcionalidad fue incorporada mediante un mecanismo de integración propio del memorista responsable, sin un con-

trato de interfaz común, lo que explica la heterogeneidad de mecanismos de carga identificada en el diagnóstico.

La evaluación de Ibaceta Jaña (2023) identificó fallas internas (consistencia de los módulos entre sí) y externas, vinculadas a la interacción con el hardware y a la ausencia de documentación de uso e instalación. Entre los hallazgos más críticos se cuentan la inestabilidad de la cámara digital (E16) y la imposibilidad de detectar la parte inferior de la proyección con el eye tracker (E49), lo que compromete la validez de EDDIE como instrumento de medición del comportamiento atencional. Sus pruebas de reconocimiento de texto registraron además que la cámara detectaba el botón de resaltado de la interfaz en lugar del texto, por lo que sugirió cambiar su color (EFD2), y que la validez del reconocimiento caía del 60–100 % en las posiciones cercanas a la cámara al 10–30 % en las más lejanas, por lo que sugirió ubicar el texto en la parte superior de la interfaz (EFD4). Este diagnóstico constituye la línea base contra la cual se contrastan los resultados de la reconstrucción (Capítulo 5).

2.2.2 Sistemas de lectura aumentada similares

EDDIE no existe en un vacío tecnológico: hay sistemas de lectura aumentada comparables y plataformas que han enfrentado problemas de modernización arquitectónica. Entre los referentes más destacados se encuentran:

- **DigitalDesk** (Wellner, 1993): sistema pionero que introdujo el concepto de entorno aumentado por computador, con una cámara de video y un proyector montados sobre un escritorio convencional. Sus aplicaciones incluyen una calculadora que lee por OCR los números que el usuario señala en un informe impreso, un traductor que proyecta definiciones de palabras extranjeras y PaperPaint, que permite copiar bocetos en papel como objetos digitales proyectados (Figura 2.1).
- **EnhancedDesk** (Koike et al., 2001): evoluciona el escritorio aumentado resolviendo la sensibilidad a la iluminación con una cámara infrarroja que sigue los dedos en tiempo real (25–30 cuadros por segundo) sin importar los colores proyectados sobre la piel. Su aplicación principal es un libro de texto interactivo en que códigos 2D impresos activan simulaciones que el estudiante manipula con las manos (Figura 2.2).
- **AR-PaperSync** (Miah and Kong, 2024): sistema de realidad aumentada móvil que integra artículos científicos impresos y digitales. Permite escanear el título

de un artículo impreso para ver capas de información con citas y referencias, y sincroniza las notas de lectura entre el móvil, el escritorio y el papel (Figura 2.3).

- **Enlight** (Wander, sf): lámpara de escritorio que combina visión computacional, OCR, fuentes de datos en red y *projection mapping* para traer funciones inteligentes a libros impresos: subrayar una palabra muestra su definición y los resaltados se sincronizan con la nube. Es un prototipo conceptual de diseño de interacción, sin rastreo ocular ni consistencia bidireccional, pero comparte el principio de proyección no obstructiva (Figura 2.4).

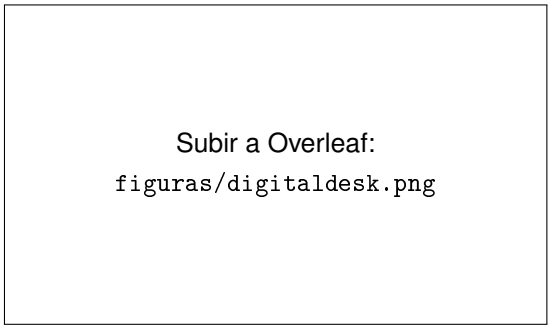


Figura 2.1: Primer prototipo de DigitalDesk.
Fuente: Wellner (1993).

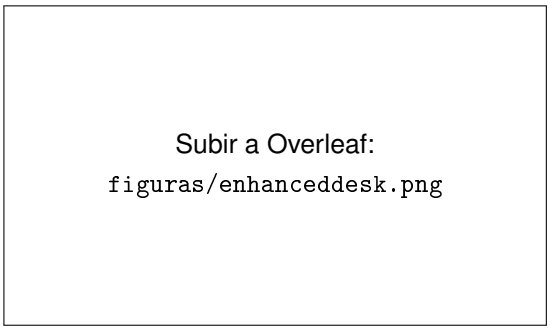


Figura 2.2: Gesto de señalamiento en EnhancedDesk.
Fuente: Koike et al. (2001).

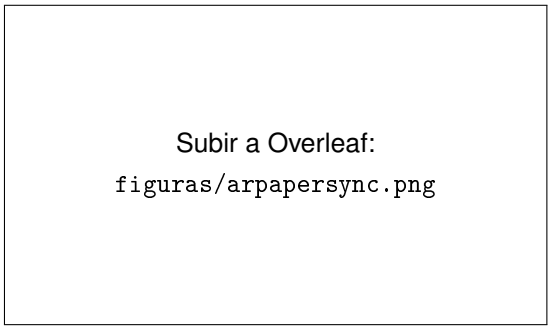


Figura 2.3: Descripción general de AR-PaperSync.
Fuente: Miah and Kong (2024).

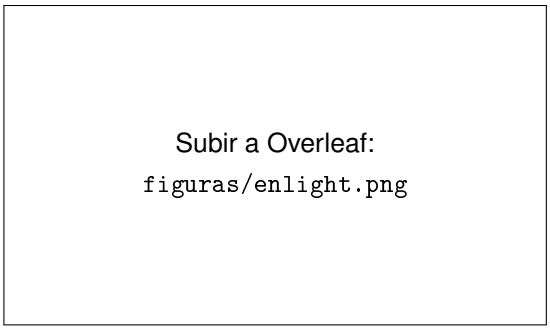


Figura 2.4: Enlight: lectura aumentada.
Fuente: Wander (sf).

La Tabla 2.2 compara estos sistemas con EDDIE en las cuatro características que lo definen. EDDIE es el único que combina proyección sobre papel, rastreo ocular y consistencia bidireccional con un propósito de investigación académica, lo que justifica conservarlo como plataforma en lugar de reemplazarlo por un sistema existente.

Tabla 2.2: Comparación de sistemas de lectura aumentada.

Sistema	Proyección sobre papel	Rastreo ocular	Consistencia bidireccional	Foco académico
EDDIE	Sí	Sí	Sí	Sí
DigitalDesk	Sí	No	Parcial	No
EnhancedDesk	Sí	No	No	Sí
AR-PaperSync	No (móvil)	No	Sí	Sí
Enlight	Sí	No	Parcial	No

Fuente: Elaboración propia a partir de los trabajos citados.

2.2.3 El problema de la modernización en software de investigación

Un desafío documentado en el desarrollo de software científico es que decisiones técnicas localmente óptimas, tomadas por un investigador para resolver el problema inmediato de su propia tesis, tienden a producir un sistema globalmente inconsistente cuando se acumulan a través de múltiples autores sin una arquitectura común (Ernst et al., 2026). El fenómeno se ha documentado en sistemas de mayor escala, como plataformas de bioinformática y biología computacional, donde la falta de coordinación entre grupos de desarrollo genera herramientas difíciles de integrar entre sí (Orozco Solano, 2017).

La literatura converge en que la solución recurrente es adoptar arquitecturas modulares con interfaces estandarizadas que separen la lógica de investigación de la infraestructura técnica (Bustio Martínez et al., 2013; Glatard et al., 2017). En los *science gateways*, plataformas que integran instrumentos y flujos de trabajo científicos, esta separación se implementa habitualmente con una capa de abstracción de hardware y un catálogo de plugins cargados dinámicamente a través de un único mecanismo, lo que permite aislar la falla de un componente sin comprometer el resto (Glatard et al., 2017). Un núcleo liviano con contratos bien definidos reduce el tiempo dedicado a resolver problemas comunes de integración y concentra el esfuerzo en los algoritmos de cada dominio (Bustio Martínez et al., 2013).

2.2.4 Multimodal Observer como referente arquitectural

Un antecedente crítico es la evolución de Multimodal Observer (MO), aplicación del mismo laboratorio InTeractiOn para la captura y el análisis de datos multimodales como movimientos oculares, bioseñales y video. MO enfrentó un problema estructural-

mente idéntico al de EDDIE: las sucesivas versiones de su núcleo, desarrolladas por distintos tesistas, generaron incompatibilidades que dejaron inoperativos la mayoría de sus plugins de visualización y análisis. La solución fue reimplementar el núcleo en Python con una arquitectura de plugins desacoplada, que resultó viable en el contexto académico y produjo un sistema validado con métricas objetivas de calidad de código, rendimiento y aceptación (Figueroa Contreras, 2025). Esta experiencia es el antecedente empírico más relevante para la presente propuesta, por la similitud del problema y por compartir el contexto institucional y la línea de investigación (Figura 2.5).

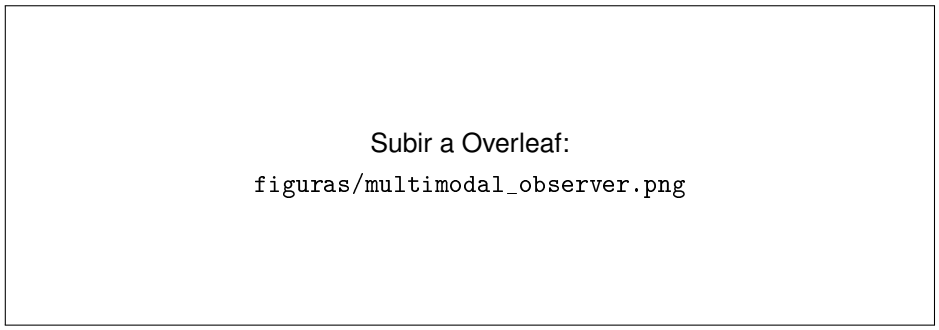


Figura 2.5: Multimodal Observer: vista principal.
Fuente: Figueroa Contreras (2025).

CAPÍTULO 3. ANÁLISIS

Este capítulo corresponde a la etapa *Plan* del ciclo PDCA y al primer objetivo específico. Presenta el diagnóstico de la arquitectura en C#, la captura de requerimientos de la nueva versión, el análisis de factibilidad y los prototipos desechables con los que se evaluó la migración de cada función crítica.

3.1 DIAGNÓSTICO

3.1.1 Reclasificación de los errores de EDDIE-2023

Ibaceta Jaña (2023) registró 67 errores al evaluar EDDIE con MECIVA PS, organizados según el módulo en que aparecieron (consistencia, rastreo ocular, documento físico, otros). Esa organización indica *dónde* se manifiesta cada falla, pero no *por qué* ocurre, que es lo que determina si una reconstrucción del núcleo puede corregirla. Por ello se reclasificaron los 67 errores según su causa raíz (Tabla 3.1; el detalle por error está en el Anexo A).

Tabla 3.1: Reclasificación por causa de los 67 errores documentados por Ibaceta Jaña (2023).

Causa	**N.º**	**%**	**Ejemplos y relación con la reconstrucción**
Carga y despliegue	16	23,9	DLL o carpeta de plugins faltante (E50), rutas absolutas de desarrolladores anteriores, ensamblados copiados a mano. Los corrige el diseño: un único cargador y un único archivo de configuración.
Práctica de código	26	38,8	Variables con valor nulo, excepciones no capturadas, acciones sobre un PDF no cargado (E11). Los corrige el aislamiento de fallos del orquestador y las validaciones de cada plugin.
Hardware y sensores	9	13,4	Eye tracker no reconocido (E15), cámara inestable (E16), zona inferior de la proyección no detectable (E49). Requieren el laboratorio.
Servicios externos	8	11,9	APIs de búsqueda con credenciales vencidas. Se reemplazan por un modelo de lenguaje.
Interfaz e idioma	3	4,5	Mensajes en un idioma distinto (E14).
Infraestructura de laboratorio	3	4,5	Equipo, montaje y red de la sala. Fuera del alcance del software.
Documentación	2	3,0	Sin manual de instalación ni de uso (E12, E13).
Total	**67**	**100**	

Fuente: Elaboración propia a partir de Ibaceta Jaña (2023).

La reclasificación muestra que las dos causas más frecuentes, práctica de código (26) y carga y despliegue (16), suman 42 errores (62,7 %) que dependen de la arquitectura y no del hardware. Son las que una reconstrucción del núcleo puede atacar de forma estructural. Los errores de hardware e infraestructura (12) solo pueden verificarse en el laboratorio, y los de servicios externos (8) se resuelven cambiando de proveedor.

3.1.2 Arquitectura en C#

EDDIE-2023 es una solución de Visual Studio con 26 proyectos y unas 45 000 líneas de C#. La aplicación principal (`AugmentedReadingApp`) contiene la interfaz WinForms, la proyección y la lógica de coordinación, y carga los módulos de tres formas

distintas (Figura 3.1): los plugins de consistencia con `Assembly.LoadFrom` desde una carpeta que debía existir en el equipo, los de rastreo ocular en un `AppDomain` subordinado mediante una clase intermedia `MarshalByRefObject`, y los de búsqueda con `Assembly.LoadFile`. Ninguno de los tres comparte una interfaz común con los otros, por lo que cada nuevo módulo agregó su propio punto de falla.

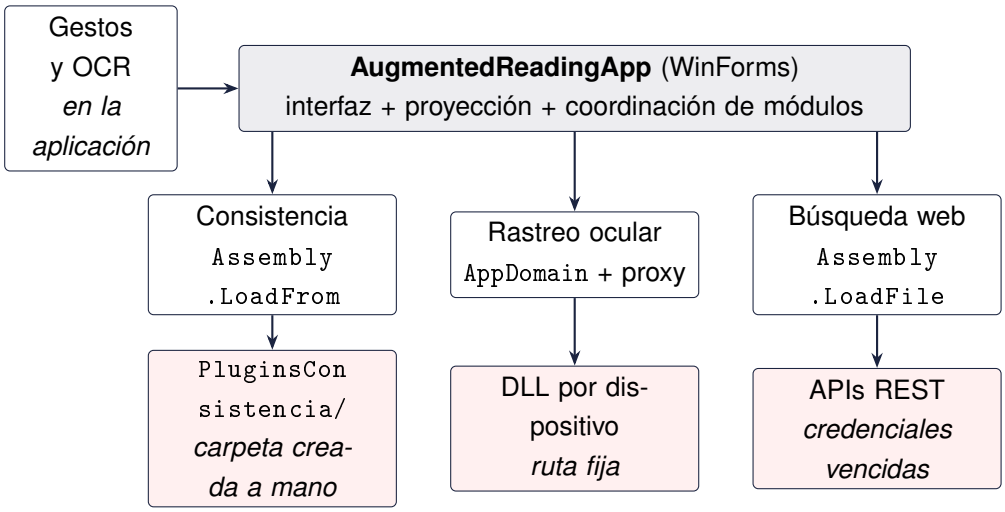


Figura 3.1: Arquitectura de EDDIE-2023: tres mecanismos de carga independientes.

Fuente: Elaboración propia a partir del repositorio EDDIE-2023.

3.2 CAPTURA DE REQUERIMIENTOS

La captura de requerimientos tuvo por objetivo definir qué debe hacer la nueva versión de EDDIE y bajo qué condiciones de calidad. El punto de partida fue el sistema legado, ubicado en el repositorio EDDIE-2023. El proceso persiguió dos metas complementarias: conservar las funcionalidades de interacción físico-digital que el sistema ya ofrecía, y resolver de manera estructural los errores que dependen de la arquitectura (Sección 3.1).

3.2.1 Fuentes y etapas del proceso

Revisión de requerimientos e historial legado. Se revisaron los requerimientos de los trabajos de titulación previos (Carrasco Madriaga, 2017; Contreras Velásquez, 2019; Montanares Mundaca, 2019; Pezoa Riquelme, 2020; Estay Vera, 2020; Santibáñez Espinoza, 2020), reunidos y verificados por Ibaceta Jaña (2023). Esta revisión estableció la línea base de capacidades que debían mantenerse (trazado de resaltados, lectura de QuadPoints, notas adhesivas, rastreo ocular) y las

distinguió de las que requerían reestructuración (carga dinámica de DLL, manejo de valores nulos). Se conservaron los identificadores originales (por ejemplo RFCO12 o RFP1) para mantener la trazabilidad.

Directrices del profesor guía y diagnóstico arquitectónico. Se integraron las prioridades surgidas del diagnóstico: desacoplar el núcleo de las bibliotecas del sistema legado (PyMuPDF en lugar de iTextSharp, OpenCV en lugar de Emgu CV, PySide6 en lugar de WinForms) e incorporar un orquestador central.

Referentes de arquitectura. Multimodal Observer (Figueroa Contreras, 2025) aportó la estructura de núcleo, contrato `PluginBase`, cargador por `importlib` y validación con dispositivos simulados. EDDIE ya contemplaba una arquitectura modular para incorporar nuevos módulos (RFCO12, Contreras Velásquez, 2019) y plugins por dispositivo de rastreo ocular (Estay Vera, 2020); la nueva versión generaliza esa idea a todos los módulos.

Hallazgos de la evaluación previa. Los hallazgos de Ibaceta Jaña (2023) sobre la interfaz proyectada (EFD2, EFD4, E49) se tradujeron en requisitos de interfaz (U1–U3 y RNF-06).

Especificación verificable. Cada requisito se redactó de modo que pudiera verificarse con una prueba automatizada (pytest), un prototipo o una prueba en el laboratorio, y cada avance indica los requisitos que cubre (Apéndice A).

3.2.2 Requerimientos funcionales

La Tabla 3.2 presenta los requerimientos funcionales agrupados por módulo. Los requisitos con identificador N son nuevos y corresponden al núcleo y a la capa de abstracción de hardware; los restantes provienen de las memorias anteriores y conservan su identificador original, y los requisitos U provienen del rediseño de la interfaz.

Tabla 3.2: Requerimientos funcionales de EDDIE por módulo.

ID	Requerimiento	Origen
Núcleo y orquestador		
N1	Cargar todos los plugins declarados en `config/plugins.json` con un único mecanismo (`importlib`).	Nuevo (núcleo)

ID	Requerimiento	Origen
N2	Aislar las fallas de carga, inicialización y ejecución de un plugin sin detener el resto.	Nuevo (núcleo)
N3	Inyectar el sensor de cámara declarado a los plugins que consumen frames.	Nuevo (núcleo)
N4	Conmutar cada dispositivo entre implementación real y simulada (`DeviceFactory`).	Nuevo (HAL)
N5	Invocar operaciones de un plugin bajo demanda, con aislamiento de errores y medición de latencia.	Nuevo (núcleo)
N6	Reiniciar o detener un plugin en ejecución y reconectar a sus consumidores.	Nuevo (núcleo)
N7	Registrar una traza de ejecución (JSONL) con cada acción y su latencia.	Nuevo (núcleo)
N8	Persistir los ajustes del usuario con las mismas claves de `Settings2` del C#.	Nuevo (núcleo)
RFCO12	Arquitectura modular para incorporar nuevos módulos/dispositivos.	Contreras (2019)
Reconocimiento de texto		
FD1	Reconocimiento del texto resaltado en el documento físico (OCR).	Carrasco (2017) RF-CA2
Consistencia físico-digital		
C1	Figuras del documento físico reflejadas en el digital.	Pezoa (2020) RFP1
C2	Comentarios (nota adhesiva) del físico reflejados en el digital.	Pezoa (2020) RFP2
C3	Marcador físico (banderilla azul) reflejado en el digital.	Pezoa (2020) RFP3
C5	Figuras de una página digital reflejadas sobre el papel (proyección).	Pezoa (2020) RFP5
C6	Comentarios de una página digital reflejados sobre el papel.	Pezoa (2020) RFP6
C7	Listar los marcadores del documento digital.	Pezoa (2020) RFP7
C8	Activar/desactivar las figuras proyectadas sobre el papel.	Pezoa (2020) RFP8
C9	Eliminar un marcador del documento digital.	Pezoa (2020) RFP9
C10	Eliminar un comentario del documento digital.	Pezoa (2020) RFP10

ID	Requerimiento	Origen
C11	Resaltar en el papel y reflejarlo en el PDF.	Carrasco (2017) RF-CA4
C12	Resaltar en el PDF y reflejarlo en el papel.	Carrasco (2017) RF-CA5
Interacción gestual		
G1	Reconocimiento gestual de la mano con cámara USB.	Contreras (2019) RFCO10
G2	Reconocimiento de la posición de un lápiz de color con cámara USB.	Contreras (2019) RFCO11
G3	Seleccionar con el gesto un fragmento y capturarlo.	Contreras (2019) RFCO5
Rastreo ocular		
RO1–RO7	Rastreo ocular: seguir la mirada, retícula, interacción por mirada, registro CSV, varios dispositivos, módulo de registro común y selección del dispositivo.	Estay (2020) RFE1–RFE7
Búsqueda y voz		
B1–B13	Búsqueda: enciclopedia, definiciones, traducción, video, imágenes, selección de proveedor, visualizar/ocultar resultados, historial, voz y lector.	Montanares (2019) RFM1–RFM13
Interfaz		
RFS1–RFS8	Interfaz: acceso a lectura aumentada, búsqueda, consistencia y rastreo; configurar, navegar, guardar y reiniciar ajustes.	Santibáñez (2020)
U1	Mostrar al abrir los pasos para preparar la sesión de lectura con el estado real de cada módulo.	Nuevo (interfaz)
U2	Fijar los puntos de calibración gestual desde la lectura actual del sensor.	Nuevo (interfaz)
U3	Informar los mensajes con avisos no modales que no detienen la lectura.	Nuevo (interfaz)

Fuente: Elaboración propia a partir de los trabajos citados en la columna Origen.

3.2.3 Requerimientos no funcionales

La Tabla 3.3 especifica los ocho requerimientos no funcionales. Cada uno tiene un criterio de aceptación medible, que se verifica en el Capítulo 5. El umbral de latencia de RNF-01 se deriva de la frecuencia de muestreo de los eye trackers: a 60 Hz llega una muestra cada 16,6 ms, y el ciclo del orquestador no debe consumir más de la mitad de ese período para no perder muestras.

Tabla 3.3: Requerimientos no funcionales y su criterio de aceptación.

ID	Requerimiento y criterio de aceptación
RNF-01	Latencia orquestador→plugin $\leq$ 8,3 ms en el percentil 95 (50 % de una muestra a 60 Hz).
RNF-02	Aislamiento de fallos: un plugin que falla no detiene el sistema y queda registrado.
RNF-03	Cobertura de líneas de las pruebas automatizadas $\geq$ 80 %.
RNF-04	Extensibilidad: un plugin nuevo se agrega con una clase y una entrada en `plugins.json`, sin tocar el núcleo.
RNF-05	Validación sin laboratorio: todo el sistema se puede ejecutar y probar con dispositivos simulados.
RNF-06	Interfaz: conserva todos los controles y flujos de EDDIE-2023 (mismo nombre y comportamiento), en español, con los botones de la proyección en la franja superior, la zona de lectura y las cruces en su posición original y sin amarillo fuera del resaltado (Ibaceta, 2023).
RNF-07	Plataforma: Windows 11, Python $\geq$ 3.11, sin dependencias nativas de 32 bits.
RNF-08	Sin APIs web vencidas: los servicios de búsqueda los responde un LLM (Gemini 2.5 Flash) y la clave no se versiona.

Fuente: Elaboración propia.

Varios de estos requerimientos responden a errores documentados en el sistema legado. RNF-02 y RNF-04 responden a la carga de plugins como archivos `.dll` que debían compilarse y ubicarse manualmente en carpetas específicas, origen de errores como la ausencia de la carpeta `PluginsConsistencia` (E50). RNF-02 responde también a las fallas por variables con valor nulo, que en C# detenían la aplicación. RNF-05 permite verificar todo el software antes de usar el laboratorio, cuya disponibilidad es limitada. RNF-06 responde a los hallazgos de usabilidad EFD2 y EFD4 y al error de idioma E14, y RNF-08 a los servicios de búsqueda con credenciales vencidas.

3.3 FACTIBILIDAD

3.3.1 Factibilidad técnica

La factibilidad técnica depende de que cada función de EDDIE-2023 tenga un equivalente en el ecosistema Python que entregue el mismo resultado. Para las funciones de bajo riesgo (consultas REST, registro de eventos, marcadores de PDF) basta con verificar que la biblioteca existe y está mantenida. Para las funciones críticas, cuyo comportamiento depende de algoritmos de visión, de protocolos de dispositivos o de la arquitectura, se construyeron prototipos desechables (Sección 3.4). Las bibliotecas elegidas (OpenCV, Tesseract, PyMuPDF, PySide6) son de código abierto, tienen versiones para Windows 11 de 64 bits y se instalan con `pip`, lo que además elimina las dependencias nativas de 32 bits del sistema legado.

El riesgo técnico principal es que OpenCV y Emgu CV no producen resultados idénticos con los mismos parámetros, por lo que los umbrales de color y de forma deben recalibrarse. Los prototipos permiten medir ese efecto antes de la integración.

3.3.2 Factibilidad económica

Todo el software utilizado es libre o gratuito para uso académico. El hardware (cámaras, proyector, GazePoint y TheEyeTribe) ya existe en el laboratorio. El único costo variable es el modelo de lenguaje que responde las búsquedas, cuyo nivel gratuito es suficiente para sesiones de laboratorio; la clave se guarda fuera del repositorio (RNF-08).

3.3.3 Factibilidad operativa

La operación depende del acceso a la sala de experimentación, que se comparte con otros proyectos. Para que el desarrollo no dependa de ese acceso, la capa de abstracción de hardware permite ejecutar y probar el sistema completo con dispositivos simulados (RNF-05), y el laboratorio se reserva para la verificación con el hardware real.

3.4 PROTOTIPADO

Esta sección presenta la fase de prototipado, cuyo propósito es reducir la incertidumbre técnica antes del diseño e implementación definitivos. Dado que EDDIE-2023

fue desarrollado en C# y depende de bibliotecas específicas de Windows, es necesario comprobar de manera temprana y con bajo costo que sus funciones esenciales pueden reproducirse en Python. Con este fin se construyeron prototipos desechables (*throwaway*), piezas de software acotadas cuyo valor está en los hallazgos que entregan y no en el código que producen. Cada prototipo se evaluó con tres criterios: (i) equivalencia funcional con la implementación en C#, (ii) disponibilidad de bibliotecas Python adecuadas y (iii) cumplimiento de los requerimientos asociados (Tablas 3.2 y 3.3).

3.4.1 Inventario de funciones

Como paso previo se inventariaron las 28 funciones del código original y se agruparon en siete subsistemas (Tablas 3.4 a 3.10). Cada tabla indica la función, su ubicación en EDDIE-2023, el hardware que requiere, la decisión de prototiparla y la tecnología Python que la reemplaza.

Tabla 3.4: Funciones del subsistema de procesamiento de imágenes y visión computacional.

ID	Función	EDDIE-2023	Hardware	¿Prototipar? / Tecnología
F1	Enumeración e inicialización de cámaras	`CameraActivity.cs`	Cámaras de texto y gestos	Sí. OpenCV `VideoCapture` (MSMF con respaldo `CAP_ANY`) y nombres de dispositivo por PowerShell/WMI.
F2	Preprocesamiento y binarización (Otsu)	`ColorRecognition.cs`	Cámara de texto	Sí. OpenCV `cvtColor`, `GaussianBlur`, `threshold`.
F3	Reconocimiento óptico de caracteres	`OCRProcess.cs`	Ninguno	Sí. `pytesseract` (Tesseract 5, `spa+eng`).
F4	Detección automática de páginas	`PageDetectionSettings.cs`	Cámara de texto y proyector	Sí (solo factibilidad; fuera del MVP). OpenCV `Canny`, `findContours`, `approxPolyDP`.

Fuente: Elaboración propia a partir del repositorio EDDIE-2023.

Tabla 3.5: Funciones del subsistema de reconocimiento gestual.

ID	Función	EDDIE-2023	Hardware	¿Prototipar? / Tecnología
F5	Seguimiento de lápiz de color (Color Pen)	`ColorPenRecognition`	Cámara de gestos	Sí. OpenCV `inRange`.
F6	Segmentación de mano y apuntado (Hand Skin)	`HandSkinRecognition` / `YCrCbSkinDetector.cs`	Cámara de gestos	Sí. OpenCV YCrCb, contornos y defectos de convexidad.
F7	Mapeo de coordenadas cámara → proyección	`GestureRecognitionActivity.cs`	Cámara de gestos y proyector	Sí. OpenCV `findHomography`.
F8	Reconocimiento gestual 3D	`LeapMotionPlugin.cs`	Leap Motion	No (sensor fuera del alcance).
F9	Mouse como alternativa (Mouse Recognition)	`MousePlugin.cs`	Mouse	Sí. Eventos de ventana de OpenCV y cursor del sistema operativo.

Fuente: Elaboración propia a partir del repositorio EDDIE-2023.

Tabla 3.6: Funciones del subsistema de rastreo ocular.

ID	Función	EDDIE-2023	Hardware	¿Prototipar? / Tecnología
F10	Comunicación con el eye tracker	Plugins GazeCloud (WebSocket, puerto 3000), EyeTribe (TCP 6555) y GazePoint (Open Gaze API, TCP 4242)	Eye tracker	Sí. `socket` y `asyncio` con mensajes JSON (EyeTribe) y XML (GazePoint).
F11	Carga y aislamiento de plugins de rastreo	`IntermediateClass.cs` (`AppDomain`)	Ninguno	Sí. `importlib`, mismo mecanismo para todos los plugins.
F12	Clic por permanencia de la mirada (*dwell*)	`MouseControl.cs`	Eye tracker	Sí. Filtro temporal con zona de radio fijo (0,6 s, 45 px).
F13	Retícula de mirada proyectada	`ReticleDrawing.cs`	Eye tracker y proyector	Sí. Dibujo con mezcla alfa (OpenCV en el prototipo; Qt en la versión final).

Fuente: Elaboración propia a partir del repositorio EDDIE-2023.

Tabla 3.7: Funciones del subsistema de consistencia físico-digital.

ID	Función	EDDIE-2023	Hardware	¿Prototipar? / Tecnología
F14	Búsqueda de texto y QuadPoints en el PDF	`DigitalDocSync.cs` (iTextSharp)	Ninguno	Sí. PyMuPDF `search_for(quads=True)`.
F15	Sincronización de resaltado papel → PDF	`DigitalDocSync.cs`	Cámara de texto	Sí. PyMuPDF `add_highlight_annot`.
F16	Sincronización de notas adhesivas	`CommentsDP` / `CommentsPD`	Cámara de texto y proyector	Sí. OpenCV (detección HSV) y PyMuPDF `add_text_annot`.
F17	Sincronización de figuras geométricas	`FiguresDP` / `FiguresPD`	Cámara de texto y proyector	Sí. OpenCV (`approxPolyDP`, compacidad) y PyMuPDF.
F18	Sincronización de marcadores de página	`PageMarkerDP` / `PageMarkerPD`	Cámara de texto	No (operación directa de PyMuPDF).

Fuente: Elaboración propia a partir del repositorio EDDIE-2023.

Tabla 3.8: Funciones del subsistema de búsqueda.

ID	Función	EDDIE-2023	Hardware	¿Prototipar? / Tecnología
F19	Búsqueda enciclopédica	`ApiWikipedia.cs`	Internet	No. La responde el modelo de lenguaje (`IWebSearchProvider`).
F20	Definiciones	`ApiDefinicionesGoogle.cs`	Internet	No. Modelo de lenguaje.
F21	Traducción	`ApiTraduccionBing.cs`	Internet	No. Modelo de lenguaje.
F22	Búsqueda de imágenes	`ApiBusquedaImagenCloudVision.cs`	Cámara e Internet	No. Modelo de lenguaje con la imagen del recorte.
F23	Búsqueda de videos	`ApiBuscarYoutube.cs`	Internet	No. Modelo de lenguaje (sugerencia de búsqueda y enlace).

Fuente: Elaboración propia a partir del repositorio EDDIE-2023.

Tabla 3.9: Funciones del subsistema de voz.

ID	Función	EDDIE-2023	Hardware	¿Prototipar? / Tecnología
F24	Comandos de voz	`ProjectionScreenActivity2.cs` (`System.Speech`)	Micrófono	Sí. `SpeechRecognition` (motor de Google en línea; Whisper o Vosk locales como alternativa).
F25	Lectura en voz alta	`ProjectionScreenActivity2.cs` (`System.Speech`)	Parlantes	Sí. `pyttsx3` (voces SAPI5 de Windows).

Fuente: Elaboración propia a partir del repositorio EDDIE-2023.

Tabla 3.10: Funciones del subsistema de presentación, proyección y registro.

ID	Función	EDDIE-2023	Hardware	¿Prototipar? / Tecnología
F26	Navegador embebido	`ProjectionScreenActivity2.cs` (CefSharp)	Proyector	No. Resultados como texto en la proyección y enlace al navegador del sistema.
F27	Banda de resaltado proyectada	`HighlightTool.cs`	Proyector	Sí. Dibujo con mezcla alfa (OpenCV en el prototipo; Qt en la versión final).
F28	Registro de eventos y mirada	Registro CSV de cada plugin	Ninguno	No. `logging`, CSV y traza JSONL del orquestador.

Fuente: Elaboración propia a partir del repositorio EDDIE-2023.

3.4.2 Síntesis de la selección y método de medición

De las 28 funciones, 19 se prototiparon y 9 quedaron excluidas por tres razones: depender de hardware fuera del alcance (F8), tener un equivalente directo sin riesgo técnico (F18, F26, F28) o depender de un proveedor externo que se sustituyó por un modelo de lenguaje (F19 a F23).

Cada prototipo reproduce el algoritmo del sistema legado en un script independiente (carpeta `prototipos/` del repositorio). Para que sus resultados sean reproducibles, las métricas de las fichas se obtuvieron con un único script, `mediciones/medir_prototipos.py` del avance P-00, que ejecuta el algoritmo de cada prototipo sobre datos con verdad terreno conocida y escribe los valores que aparecen en este capítulo. Cada resultado indica su tipo de medición:

- [pdf]: sobre un PDF real (`borrador_tesis.pdf`, 64 páginas).

- [código]: sobre los módulos del código final.
- [sintético]: sobre imágenes o señales generadas con verdad terreno conocida (por ejemplo, una página del PDF con un destello simulado o una traza de mirada con ruido gaussiano). Mide el algoritmo, no el sensor.
- [laboratorio]: requiere la cámara, el eye tracker, el micrófono o participantes; se marca como [Pendiente: medir en el laboratorio] y se mide con el mismo script.

Las mediciones se ejecutaron el 2026-11-08 en Linux 6.18.44-fc-v310 con semilla fija. Los tiempos se midieron con `time.perf_counter()` tras 10 ejecuciones de calentamiento y se reportan como media $\pm$ desviación estándar.

3.4.3 Prototipo F1: Enumeración e inicialización de cámaras

En el sistema legado (`CameraActivity.cs`) esta función enumera las cámaras con DirectShow y asigna una a la captura de texto y otra a la de gestos. El prototipo abre cada cámara con OpenCV (backend Media Foundation con respaldo `CAP_ANY`) y obtiene los nombres reales de los dispositivos con una consulta WMI desde PowerShell, de modo que la interfaz pueda mostrar «cámara cenital» en lugar de un índice.

Tabla 3.11: Ficha de validación del prototipo: Enumeración e inicialización de cámaras.

Objetivo	Abrir, configurar y leer las cámaras de texto y de gestos desde Python, con el nombre real de cada dispositivo.
Librerías / Hardware	OpenCV (`VideoCapture`), PowerShell (`Get-CimInstance Win32_PnPEntity`) / dos cámaras USB.
Contrato destino	`ICameraSensor` (`CameraSensorPlugin` de la capa de hardware).
Requisitos asociados	N3, N4; RNF-05, RNF-07.
Resultados y hallazgos	[laboratorio] [Pendiente: FPS efectivos y latencia de `cap.read()` con las cámaras del laboratorio]. La enumeración y la conmutación a la cámara simulada se verifican con pruebas automatizadas desde P-02.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Con cada cámara del laboratorio se descartan 30 cuadros de calentamiento y se leen 300 cuadros a 640×480, midiendo cada llamada a `cap.read()`. La tasa efectiva es

$$\mathrm{FPS} = \frac{N}{\sum_{i=1}^{N} \Delta t_i}, \tag{3.1}$$

y se ejecuta con `python mediciones/medir_prototipos.py -pdf <pdf> -camara 0` (y luego `-camara 1`). La medición responde además al error E16 (Ibaceta Jaña, 2023): si la tasa cae o aparecen cuadros vacíos, el problema es del dispositivo o de su conexión y no del software.

3.4.4 Prototipo F2: Preprocesamiento y binarización

En el sistema legado (`ColorRecognition.cs`) la imagen del papel se convierte a grises, se suaviza con un filtro gaussiano y se binariza con Otsu antes del OCR. La luz del proyector sobre el papel produce destellos y sombras, por lo que el prototipo mide cuánto de la página se recupera correctamente en esas condiciones.

Tabla 3.12: Ficha de validación del prototipo: Preprocesamiento y binarización.

Objetivo	Reproducir la binarización de Otsu del sistema legado y medir su eficacia bajo la iluminación no uniforme de un proyector.
Librerías / Hardware	OpenCV (`cvtColor`, `GaussianBlur`, `threshold`, `adaptiveThreshold`).
Contrato destino	`IImageProcessor` (método `preprocess`).
Requisitos asociados	FD1; RNF-01.
Resultados y hallazgos	[sintético] Eficacia de Otsu global en las zonas afectadas: 25,8 % ($\pm$ 2,7) sobre 30 páginas; F1 de tinta 32,2 %. Umbral adaptativo gaussiano en las mismas imágenes: 96,8 %. Tiempo: 0,67 $\pm$ 0,07 ms por página de 640 px.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se renderizaron 30 páginas del PDF a 640 px de ancho y a cada una se le agregó un destello elíptico, una sombra lateral y ruido gaussiano ($\sigma = 4$). La verdad terreno es la binarización de la página limpia. La eficacia se calcula

solo sobre los píxeles de las zonas de destello o sombra:

$$\text{Eficacia} = \left(1 - \frac{FP_{\text{ruido}} + FN_{\text{corte}}}{P_{\text{afectados}}}\right) \times 100\,\%, \tag{3.2}$$

donde FP_{ruido} son los píxeles de fondo marcados como tinta, FN_{corte} los de tinta perdidos y $P_{\text{afectados}}$ el total de píxeles de esas zonas.

Hallazgo. Un umbral global no puede separar tinta y fondo cuando la iluminación varía a lo largo de la página: Otsu elige un único valor y la zona del destello queda en blanco o la de la sombra en negro. El umbral adaptativo, que calcula un valor por vecindad, recupera casi toda la página. En la versión final esto pesa menos de lo que sugiere la cifra, porque Otsu se aplica solo al recorte pequeño de la palabra resaltada y no a la página completa, pero la sensibilidad a la luz del proyector debe medirse con la cámara real (E40/E41 de Ibaceta Jaña, 2023).

3.4.5 Prototipo F3: Reconocimiento óptico de caracteres

En el sistema legado (`OCRProcess.cs`) el recorte preprocesado se transcribe con Tesseract, el mismo motor que usaban Carrasco Madriaga (2017) y Contreras Velásquez (2019). El prototipo evalúa Tesseract 5 mediante `pytesseract` con las tasas de error por carácter y por palabra.

Tabla 3.13: Ficha de validación del prototipo: Reconocimiento óptico de caracteres.

Objetivo	Transcribir texto en español e inglés con Tesseract desde Python y medir su exactitud.
Librerías / Hardware	`pytesseract` (Tesseract 5, `-oem 3 -psm 6 -l spa+eng`), OpenCV.
Contrato destino	`IImageProcessor` (método `extract_text`).
Requisitos asociados	FD1; RNF-07.
Resultados y hallazgos	[pdf] 60 bloques de 3 líneas (2405 palabras, 16353 caracteres). Con el preprocesamiento del prototipo: CER 1,46 %, WER 6,86 %. Sin preprocesamiento: CER 0,42 %, WER 0,71 %. Imagen degradada (desenfoque, brillo, ruido) con preprocesamiento: CER 2,53 %, WER 9,27 %. Tiempo: 381,1 $\pm$ 48,2 ms por bloque.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se tomaron las tres primeras líneas de 60 párrafos de texto corrido del PDF (sin tablas ni índices), se renderizaron a 150 ppp y se transcribieron con el preprocesamiento de `prototipo_f03.py` (CLAHE, Otsu, apertura 2$\times$2 y escala $\times$2). La verdad terreno es el texto que el propio PDF contiene. Las tasas se calculan con la distancia de Levenshtein:

$$\mathrm{CER} = \frac{S_c + D_c + I_c}{N_c} \times 100\,\%, \qquad \mathrm{WER} = \frac{S_w + D_w + I_w}{N_w} \times 100\,\%. \tag{3.3}$$

Hallazgo. Sobre texto limpio el preprocesamiento *empeora* el resultado: Tesseract ya binariza internamente y la apertura morfológica adelgaza los trazos finos y las tildes. El preprocesamiento se justifica solo con imágenes degradadas, como las de la cámara. Este resultado es un límite superior, porque el texto renderizado no tiene la perspectiva ni el desenfoque de una foto; la validez por palabra y posición respecto de la cámara, que Ibaceta Jaña (2023) identificó como determinante, se mide en el laboratorio [Pendiente: OCR con la cámara real, $\geq$ 10 palabras por posición].

3.4.6 Prototipo F4: Detección automática de páginas

En el sistema legado (`PageDetectionSettings.cs`) se detectan los bordes del libro para ajustar el área de proyección. La detección automática del libro quedó fuera del alcance del MVP; el prototipo se mantuvo para registrar su factibilidad.

Tabla 3.14: Ficha de validación del prototipo: Detección automática de páginas.

Objetivo	Detectar el contorno de la página en la imagen de la cámara y estimar sus cuatro esquinas.
Librerías / Hardware	OpenCV (`Canny`, `findContours`, `approxPolyDP`).
Contrato destino	Ninguno en el MVP (factibilidad para trabajo futuro).
Requisitos asociados	C11 (parcial); RNF-01.
Resultados y hallazgos	[sintético] Éxito (cuadrilátero con RMSE $\leq$ 10 px): 97,5 % de 80 escenas. RMSE de las esquinas detectadas: 2,26 $\pm$ 0,14 px.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se generaron 80 escenas de 640$\times$480 con una página del PDF sobre un escritorio oscuro con textura, girada entre $-15°$ y $+15°$, con escala de 0,9 a 1,25, leve perspectiva y brillo de 0,75 a 1,1. Las esquinas detectadas se comparan con las reales:

$$\mathrm{RMSE} = \sqrt{\frac{1}{4N}\sum_{i=1}^{N}\sum_{k=1}^{4}\left[(x_{k,i}-\hat{x}_{k,i})^2+(y_{k,i}-\hat{y}_{k,i})^2\right]}. \tag{3.4}$$

El resultado indica que el algoritmo es viable con buen contraste página–escritorio; con un escritorio claro o con la proyección encendida debe medirse en el laboratorio.

3.4.7 Prototipo F5: Seguimiento de lápiz de color

En el sistema legado (`ColorPenRecognition`) la punta roja de un lápiz se segmenta por color para usarla como puntero sobre el papel (Contreras Velásquez, 2019). El prototipo la segmenta en HSV, donde el rojo ocupa los dos extremos del tono, y toma el contorno rojo más grande.

Tabla 3.15: Ficha de validación del prototipo: Seguimiento de lápiz de color.

Objetivo	Localizar la punta roja del lápiz en la imagen de la cámara de gestos.
Librerías / Hardware	OpenCV (`cvtColor` HSV, `inRange`, `morphologyEx`, `findContours`).
Contrato destino	`IGesturePlugin` (`ColorPenPlugin`).
Requisitos asociados	G2; RNF-01, RNF-04.
Resultados y hallazgos	[sintético] Serie A (proyección sin rojo): detección 100 %, 0 detecciones erróneas por cada 100 cuadros, error 0,73 $\pm$ 0,3 px. Serie B (un recuadro rojo proyectado): detección 0 %, 99,7 erróneas por cada 100 cuadros. Tiempo: 1,93 ms por cuadro.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se generaron dos series de 300 cuadros con una página del PDF, tres recuadros de color que simulan contenido proyectado y, en 4 de cada 5 cuadros, la punta del lápiz (radio 12 px) en una posición conocida. En la serie A los recuadros tienen cualquier tono salvo rojo; en la B uno es rojo. Una detección es correcta si el centroide está a $\leq$ 15 px de la punta real:

$$e = \sqrt{(x-\hat{x})^2 + (y-\hat{y})^2}, \qquad \text{erróneas} = \frac{N_{\text{detecciones erróneas}}}{N_{\text{cuadros}}} \times 100. \tag{3.5}$$

Hallazgo. El algoritmo es exacto mientras no haya otro objeto rojo más grande que la punta. Con contenido rojo en la proyección toma ese objeto en todos los cuadros. Es el mismo tipo de problema que Ibaceta Jaña (2023) observó con el botón de resaltado, que la cámara detectaba en lugar del texto (EFD2): la proyección no debe usar el color que el sistema busca. En la interfaz nueva se evita el rojo y el amarillo fuera de su uso específico (RNF-06).

3.4.8 Prototipo F6: Segmentación de mano y apuntado

En el sistema legado (`YCrCbSkinDetector.cs`) la piel se aísla en el espacio YCrCb y la punta del índice se obtiene del contorno de la mano (Contreras Velásquez, 2019). El prototipo reproduce la máscara con OpenCV.

Tabla 3.16: Ficha de validación del prototipo: Segmentación de mano y apuntado.

Objetivo	Aislar la mano por color de piel y ubicar la punta del dedo índice.
Librerías / Hardware	OpenCV (YCrCb, HSV, `findContours`, `convexityDefects`).
Contrato destino	`IGesturePlugin` (`HandSkinPlugin`).
Requisitos asociados	G1; RNF-01.
Resultados y hallazgos	[sintético] 200 cuadros: IoU de la máscara 96,5 %; punta a $\leq$ 15 px en el 96,5 % de los cuadros, error medio 1,2 px. Tiempo: 2,17 ms por cuadro. [Pendiente: precisión con manos reales, distintos tonos de piel y distancias]

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se generaron 200 cuadros con una silueta de mano (palma e índice) en tonos de piel aleatorios sobre una página del PDF. La máscara es $\mathrm{Cr} \in [133, 173] \wedge \mathrm{Cb} \in [77, 127] \wedge H \leq 30$ y la punta es el punto más alto de la máscara. La calidad de la máscara M respecto de la real G se mide con

$$\mathrm{IoU} = \frac{|M \cap G|}{|M \cup G|} \times 100\,\%. \tag{3.6}$$

Al ser siluetas sintéticas, el resultado valida el algoritmo pero no su robustez ante la diversidad de manos, que se evalúa con participantes.

3.4.9 Prototipo F7: Mapeo de coordenadas

En el sistema legado (`GestureRecognitionActivity.cs`) las coordenadas de la cámara se llevan a la resolución del proyector con una calibración de cuatro puntos. El prototipo estima una homografía con OpenCV y mide el error de reproyección.

Tabla 3.17: Ficha de validación del prototipo: Mapeo de coordenadas (homografía).

Objetivo	Transformar posiciones de la cámara de gestos al plano de la proyección con una calibración de cuatro puntos.
Librerías / Hardware	OpenCV (`findHomography`, `perspectiveTransform`).
Contrato destino	`IGesturePlugin` y módulo `spatial_calibration`.
Requisitos asociados	G3, U2; RNF-01.
Resultados y hallazgos	[sintético] 300 calibraciones: error de reproyección 1,53 $\pm$ 0,51 px (percentil 95: 2,46 px) con un error de clic de $\sigma = 1{,}5$ px.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. En cada ensayo se simula una inclinación de la cámara (esquinas desplazadas hasta $\pm$80 px) y cuatro clics en las esquinas con error gaussiano $\sigma = 1{,}5$ px. Con la homografía estimada se reproyectan los 21 puntos interiores de una grilla de 5$\times$5:

$$e_{\text{reproy}} = \frac{1}{21} \sum_{j=1}^{21} \left\| H\, p_j - p_j^{*} \right\|_2 . \tag{3.7}$$

El error resultante es del orden del error de clic, lo que indica que la homografía no agrega error propio y que la precisión depende del cuidado al calibrar. Con la lámina patrón y la cámara reales se mide en el laboratorio [Pendiente: error de reproyección con la lámina patrón].

3.4.10 Prototipo F9: Mouse como alternativa

En el sistema legado (`MousePlugin.cs`) el mouse sustituye a la cámara de gestos. Su relevancia se explica porque EDDIE-2023 no podía inicializarse sin cámara disponible ni funcionaba correctamente sin dos cámaras (E7 y E8, Ibaceta Jaña, 2023): un módulo de respaldo solo es útil si el núcleo permite usarlo cuando falta el sensor.

Tabla 3.18: Ficha de validación del prototipo: Mouse como alternativa.

Objetivo	Usar el mouse como dispositivo de interacción cuando no hay cámara de gestos.
Librerías / Hardware	OpenCV (`setMouseCallback`) y cursor del sistema operativo.
Contrato destino	`IGesturePlugin` (`MousePlugin`).
Requisitos asociados	G1–G3 (alternativa), N2; RNF-02, RNF-05.
Resultados y hallazgos	[laboratorio] [Pendiente: latencia de 1 000 eventos en el PC con interfaz gráfica]. Que el sistema arranque y siga funcionando sin cámara se verifica con pruebas automatizadas (aislamiento de fallos, P-01).

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se ejecuta `prototipo_f09.py` en el PC y se registran 1 000 eventos (movimiento, clic y arrastre) con la marca de tiempo del evento y la del *callback*: $\Delta t = t_{\text{callback}} - t_{\text{evento}}$.

3.4.11 Prototipo F10: Comunicación con el eye tracker

En el sistema legado cada eye tracker se comunica por red local: GazeCloud por WebSocket (puerto 3000), TheEyeTribe por TCP con mensajes JSON (puerto 6555) y GazePoint por la Open Gaze API sobre TCP con mensajes XML (puerto 4242) (Estay Vera, 2020). El prototipo implementa el cliente en Python y mide la regularidad de la recepción.

Tabla 3.19: Ficha de validación del prototipo: Comunicación con el eye tracker.

Objetivo	Recibir muestras de mirada a 60 Hz por socket y decodificarlas sin pérdidas.
Librerías / Hardware	`socket`, `asyncio`, `json` / GazePoint, TheEyeTribe, GazeCloud.
Contrato destino	`IEyeTracker` (`EyeTrackerPlugin`).
Requisitos asociados	RO1, RO5; RNF-01, RNF-02.
Resultados y hallazgos	[sintético] 3000 de 3000 muestras recibidas (pérdida 0 %), 60 Hz, *jitter* 0,45 ms. [Pendiente: latencia y pérdida con GazePoint y TheEyeTribe]

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Un servidor local envía 3000 muestras a 60 Hz con el formato JSON del protocolo de TheEyeTribe y el cliente las decodifica como el prototipo. El *jitter* es la desviación estándar del intervalo entre recepciones:

$$\text{Jitter} = \sqrt{\frac{1}{M-1}\sum_{k=1}^{M}\left(\Delta t_k - \overline{\Delta t}\right)^2}. \tag{3.8}$$

El enlace es local (127.0.0.1), por lo que no incluye la latencia del dispositivo; confirma que el cliente en Python no pierde muestras ni introduce retardos.

3.4.12 Prototipo F11: Carga dinámica de plugins

En el sistema legado los plugins de rastreo ocular se cargan en un `AppDomain` subordinado mediante una clase `MarshalByRefObject`, uno de los tres mecanismos de carga del sistema. El prototipo reemplaza los tres por `importlib`, el mecanismo que usa el `PluginLoader` final.

Tabla 3.20: Ficha de validación del prototipo: Carga dinámica de plugins.

Objetivo	Cargar e instanciar cualquier plugin con un único mecanismo, sin compilar ni copiar archivos.
Librerías / Hardware	`importlib.util` (`spec_from_file_location`, `exec_module`).
Contrato destino	`PluginBase` / `PluginLoader`.
Requisitos asociados	N1, N2, RFCO12; RNF-02, RNF-04.
Resultados y hallazgos	[código] 240 cargas de los 8 módulos de plugins del código final: 0 fallas, 0,36 $\pm$ 0,79 ms por carga.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Cada uno de los 8 módulos se cargó 30 veces con un nombre de módulo nuevo para evitar la caché, y se midió el tiempo medio $\bar{t} = \frac{1}{N}\sum_{i=1}^{N} t_i$. Las bibliotecas que importan los módulos ya estaban cargadas, por lo que el tiempo corresponde al mecanismo y no a OpenCV o PyMuPDF. No existe telemetría de carga de EDDIE-2023 con que comparar este valor, por lo que la comparación con `AppDomain` es cualitativa: el mecanismo nuevo no requiere compilar, copiar DLL ni crear carpetas, que es lo que originaba los errores de carga y despliegue (Tabla 3.1).

3.4.13 Prototipo F12: Clic por permanencia de la mirada

En el sistema legado (`MouseControl.cs`) un control se activa cuando la mirada permanece en una zona durante un tiempo. El prototipo usa los parámetros de esa implementación, una zona de radio $R = 45$ px anclada al primer punto y un tiempo $T = 0{,}6$ s, y mide el compromiso entre activaciones voluntarias y accidentales (toque de Midas, Jacob, 1990).

Tabla 3.21: Ficha de validación del prototipo: Clic por permanencia de la mirada.

Objetivo	Medir la tasa de activaciones voluntarias y accidentales del *dwell* durante una lectura.
Librerías / Hardware	NumPy / eye tracker.
Contrato destino	`IEyeTracker` (`GazeSession` y pantalla de proyección).
Requisitos asociados	RO3; RNF-01.
Resultados y hallazgos	[sintético] Con $T = 0{,}6$ s, $R = 45$ px y ruido $\sigma = 10$ px: 100 % de las 30 miradas voluntarias activan el control y hay 3,5 activaciones accidentales por minuto de lectura. Con $\sigma = 5$ px: 100 % y 7,9/min. Con $\sigma = 15$ px: 50 % y 0,3/min.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se generó una traza de 10 minutos a 60 Hz que simula lectura (fijaciones log-normales de 150–450 ms, avances de 25–60 px, 12 % de regresiones) e incluye una mirada voluntaria de 1,2 s sobre un botón cada 15–25 s. Una activación es accidental si ocurre fuera de una mirada voluntaria:

$$\text{accidentales/min} = \frac{N_{\text{activaciones fuera de miradas voluntarias}}}{t_{\text{lectura}}}. \tag{3.9}$$

La Tabla 3.22 muestra el barrido de T y R con $\sigma = 10$ px.

Tabla 3.22: Barrido de los parámetros del clic por permanencia ($\sigma = 10$ px).

T (s)	R (px)	Voluntarias (%)	Accidentales/min
0,4	25	33,3	1
0,4	35	76,7	10,5
0,4	45	100	29,3
0,5	25	26,7	0
0,5	35	76,7	1,4
0,5	45	100	9,6
0,6	25	13,3	0
0,6	35	66,7	0,4
0,6	45	100	3,5
0,8	25	3,3	0
0,8	35	50	0
0,8	45	93,3	0,3

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Hallazgo. El radio domina el resultado: con 25 px casi ninguna mirada voluntaria se sostiene dentro de la zona, porque el ruido del sensor la saca antes de completar el tiempo, mientras que con 45 px todas lo logran. El tiempo controla las activaciones accidentales: pasar de 0,6 a 0,8 s las reduce de 3,5 a 0,3 por minuto con una pérdida menor de activaciones voluntarias. El resultado también depende del ruido: con menos ruido las fijaciones de lectura permanecen más tiempo dentro de la zona y aumentan las accidentales, y con más ruido se pierden voluntarias. Por eso los parámetros deben calibrarse con el eye tracker real y con lectores [Pendiente: calibración de T y R con GazePoint y participantes].

3.4.14 Prototipo F13: Retícula de mirada proyectada

En el sistema legado (`ReticleDrawing.cs`) se dibuja sobre la proyección una retícula en el punto que el lector mira (Estay Vera, 2020). El prototipo mide el costo de dibujarla.

Tabla 3.23: Ficha de validación del prototipo: Retícula de mirada proyectada.

Objetivo	Dibujar la retícula de mirada sin afectar la tasa de refresco de la proyección.
Librerías / Hardware	OpenCV (prototipo); Qt (`QPainter`) en la versión final.
Contrato destino	`IEyeTracker` y pantalla de proyección.
Requisitos asociados	RO2; RNF-01.
Resultados y hallazgos	[sintético] Dibujo: 0,82 $\pm$ 0,06 ms por cuadro de 950$\times$620 (máximo teórico 1224,6 cuadros/s). [Pendiente: latencia de extremo a extremo eye tracker $\rightarrow$ proyector]

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se dibujaron 600 cuadros con la retícula (círculo con mezcla alfa y cruz). La tasa máxima es $\mathrm{FPS}_{\mathrm{máx}} = 1000/\bar{t}_{\mathrm{ms}}$. El dibujo consume menos del 5 % del período de 16,6 ms de un proyector a 60 Hz; la latencia total incluye la del eye tracker y la del proyector, que solo se miden en el laboratorio.

3.4.15 Prototipo F14: Búsqueda de coordenadas QuadPoints

En el sistema legado (`DigitalDocSync.cs`) se busca en el PDF, con iTextSharp, el texto reconocido en el papel y se obtienen sus coordenadas para resaltarlo. El prototipo usa PyMuPDF, que entrega directamente los QuadPoints de cada coincidencia.

Tabla 3.24: Ficha de validación del prototipo: Búsqueda de coordenadas QuadPoints.

Objetivo	Ubicar en el PDF las coordenadas exactas de un término reconocido en el papel.
Librerías / Hardware	PyMuPDF (`Page.search_for(quads=True)`, `get_textbox`).
Contrato destino	`IConsistencyProvider` (`ConsistencyPlugin`).
Requisitos asociados	C11, C12; RNF-01.
Resultados y hallazgos	[pdf] 1920 consultas en 64 páginas: 1836 QuadPoints cuyo texto contiene el término (coherencia 82 %). Tiempo: 1,03 $\pm$ 0,62 ms por página.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se buscaron 10 términos 3 veces en las 64 páginas del PDF. Para cada QuadPoint devuelto se extrajo el texto de su rectángulo (ampliado 2 pt) y se comprobó que contuviera el término, sin distinguir tildes ni mayúsculas:

$$\text{Coherencia} = \frac{N_{\text{QuadPoints cuyo texto contiene el término}}}{N_{\text{QuadPoints}}} \times 100\,\%. \tag{3.10}$$

Hallazgo. Las incoherencias corresponden a palabras cortadas con guion al final de línea: PyMuPDF devuelve dos QuadPoints, uno por cada fragmento, y ninguno contiene el término completo. El resaltado resultante es correcto (cubre ambas partes), pero la búsqueda de una palabra reconocida en el papel puede fallar si el OCR la lee sin guion. Es un caso a considerar en las pruebas de consistencia del laboratorio.

3.4.16 Prototipo F15: Sincronización de resaltado

En el sistema legado el texto resaltado en el papel se marca en el PDF con una anotación translúcida. Ibaceta Jaña (2023) registró que el resaltado digital no coincidía en posición ni largo con el físico (E40, E41).

Tabla 3.25: Ficha de validación del prototipo: Sincronización de resaltado.

Objetivo	Agregar y guardar resaltados en el PDF de modo que persistan al reabrirlo.
Librerías / Hardware	PyMuPDF (`add_highlight_annot`, `save`).
Contrato destino	`IConsistencyProvider` (`ConsistencyPlugin`).
Requisitos asociados	C11; RNF-02.
Resultados y hallazgos	[pdf] 48 de 48 resaltados presentes al reabrir el archivo. Tiempo por resaltado (agregar, guardar y reabrir): 13,8 $\pm$ 1,7 ms. [Pendiente: coincidencia en posición y largo con el papel (E40/E41)]

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se agregaron 48 resaltados uno a uno sobre líneas del PDF; cada uno se guardó con escritura completa a un archivo temporal y reemplazo, y el archivo se volvió a abrir para contar los resaltados. Esta forma de guardar es la corrección del hallazgo H-02 (Sección 5.2.5): con PyMuPDF 2.1, los guardados incrementales sucesivos dañaban el archivo y se perdían anotaciones al reabrirlo. El tiempo se mide como $t = t_{\text{reapertura}} - t_{\text{inicio}}$.

3.4.17 Prototipo F16: Sincronización de notas adhesivas

En el sistema legado (Pezoa Riquelme, 2020) una nota adhesiva amarilla sobre el libro se detecta y se copia al PDF como comentario, y los comentarios del PDF se proyectan sobre el papel.

Tabla 3.26: Ficha de validación del prototipo: Sincronización de notas adhesivas.

Objetivo	Detectar notas adhesivas en la imagen de la página y ubicarlas en coordenadas del PDF.
Librerías / Hardware	OpenCV (HSV, `findContours`, `approxPolyDP`), PyMuPDF (`add_text_annot`).
Contrato destino	`IConsistencyProvider` (`ConsistencyPlugin`).
Requisitos asociados	C2, C6; RNF-01.
Resultados y hallazgos	[sintético] 100 % de 40 notas detectadas; error de ubicación en el PDF 0,68 pt (≈ 0,24 mm). [Pendiente: error con la cámara real y lectura del texto manuscrito]

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se pegaron digitalmente 40 notas cuadradas (70–110 px) en posiciones aleatorias sobre una página renderizada. Se detectaron con $H \in [18, 38]$, área mayor a 2 500 px y cuatro vértices, y su centro se convirtió a puntos del PDF:

$$\Delta = \sqrt{(x_{\text{real}} - \hat{x}_{\text{pdf}})^2 + (y_{\text{real}} - \hat{y}_{\text{pdf}})^2}. \tag{3.11}$$

El error es constante (desviación nula) porque corresponde a la cuantización del render: a 640 px de ancho un píxel equivale a 0,96 pt y el redondeo del centro en ambos ejes da $0{,}48\sqrt{2} \approx 0{,}68$ pt. Con la cámara real se suma el error de la homografía (F7).

3.4.18 Prototipo F17: Sincronización de figuras geométricas

En el sistema legado (Pezoa Riquelme, 2020) las figuras dibujadas en el papel (círculos, rectángulos y triángulos) se reconocen y se reproducen en el PDF. El prototipo clasifica cada contorno por su compacidad $C = 4\pi A/P^2$ y su número de vértices.

Tabla 3.27: Ficha de validación del prototipo: Sincronización de figuras geométricas.

Objetivo	Clasificar las figuras dibujadas y reconstruirlas en el PDF con su posición y tamaño.
Librerías / Hardware	OpenCV (`findContours`, `approxPolyDP`, `contourArea`, `arcLength`), PyMuPDF.
Contrato destino	`IConsistencyProvider` (`ConsistencyPlugin`).
Requisitos asociados	C1, C5; RNF-01.
Resultados y hallazgos	[sintético] Exactitud de clasificación: 80 % de 60 figuras (círculos y triángulos 100 %; 12 de 20 rectángulos clasificados como círculos). IoU de las figuras bien clasificadas: 90,1 $\pm$ 2,8 %.

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se dibujaron 60 figuras con trazo de 3 px (20 de cada tipo, tamaño y posición aleatorios). El prototipo clasifica como círculo si $C > 0{,}69$, luego como rectángulo si tiene 4 vértices y como triángulo si tiene 3. La fidelidad de la reconstrucción se mide con

$$C = \frac{4\pi A}{P^2}, \qquad \mathrm{IoU} = \frac{|F_{\mathrm{PDF}} \cap F_{\mathrm{real}}|}{|F_{\mathrm{PDF}} \cup F_{\mathrm{real}}|} \times 100\,\%. \tag{3.12}$$

Hallazgo. Un cuadrado tiene compacidad $\pi/4 \approx 0{,}785$, mayor que el umbral de 0,69, por lo que el prototipo, que evalúa la compacidad antes que los vértices, clasifica los rectángulos cercanos a un cuadrado como círculos. El plugin final, portado de `FiguresDP` del C#, evalúa primero el número de vértices y solo considera la compacidad cuando la relación de aspecto se aleja de 1, por lo que no hereda este defecto. Es un ejemplo de lo que el prototipado debe detectar: un error de orden en una regla simple que con figuras reales pasaría por un problema de iluminación.

3.4.19 Prototipo F24: Comandos de voz

En el sistema legado (Montanares Mundaca, 2019) el lector puede pedir una búsqueda con la voz, mediante `System.Speech`. El prototipo graba el audio del micrófono, lo transcribe con `SpeechRecognition` y lo interpreta como comando.

Tabla 3.28: Ficha de validación del prototipo: Comandos de voz.

Objetivo	Reconocer los comandos de búsqueda dictados en español.
Librerías / Hardware	`SpeechRecognition` (motor de Google en línea; Whisper y Vosk locales como alternativa), `sounddevice` / micrófono.
Contrato destino	`ISpeechProvider` (`SpeechPlugin`).
Requisitos asociados	B1–B13 (voz); RNF-02.
Resultados y hallazgos	[laboratorio] [Pendiente: exactitud y tiempo de respuesta de los 8 comandos con 3 personas]

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Cada uno de los 8 comandos del módulo de búsqueda se graba 10 veces por cada una de 3 personas en la sala del laboratorio. Se registra el comando reconocido y el tiempo desde el fin de la frase hasta el texto:

$$\text{Exactitud} = \frac{N_{\text{correctos}}}{N_{\text{emitidos}}} \times 100\,\%. \tag{3.13}$$

3.4.20 Prototipo F25: Síntesis de voz

En el sistema legado (Montanares Mundaca, 2019) los resultados de búsqueda pueden leerse en voz alta con el motor de Windows. El prototipo usa `pyttsx3`, que en Windows emplea las mismas voces SAPI5.

Tabla 3.29: Ficha de validación del prototipo: Síntesis de voz.

Objetivo	Leer en voz alta los resultados sin bloquear la interfaz.
Librerías / Hardware	`pyttsx3` (SAPI5) / parlantes.
Contrato destino	`ISpeechProvider` (`SpeechPlugin`).
Requisitos asociados	B1–B13 (lector); RNF-02.
Resultados y hallazgos	[laboratorio] [Pendiente: palabras por minuto y bloqueo de la interfaz en el PC]

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se leen 10 textos de 100 palabras en un hilo aparte y se mide la duración de cada lectura, $\mathrm{WPM} = N_{\text{palabras}}/t_{\text{min}}$, verificando que la interfaz siga respondiendo durante la lectura.

3.4.21 Prototipo F27: Banda de resaltado proyectada

En el sistema legado (`HighlightTool.cs`) se proyecta una banda translúcida sobre la línea que se lee. El prototipo dibuja la banda con mezcla alfa y mide su costo.

Tabla 3.30: Ficha de validación del prototipo: Banda de resaltado proyectada.

Objetivo	Superponer una banda translúcida sobre el texto sin afectar el refresco de la proyección.
Librerías / Hardware	OpenCV (prototipo); Qt (`QPainter`) en la versión final.
Contrato destino	Pantalla de proyección.
Requisitos asociados	C12; RNF-06.
Resultados y hallazgos	[sintético] Dibujo con $\alpha = 0{,}3$: 0,58 $\pm$ 0,07 ms por cuadro de 900$\times$600. [Pendiente: contraste del texto bajo la banda con luxómetro]

Fuente: Elaboración propia, con `mediciones/medir_prototipos.py` (P-00).

Protocolo de validación. Se dibujaron 600 cuadros con la banda (rectángulo amarillo y mezcla alfa 0,30). La legibilidad del texto bajo la luz proyectada se mide en el laboratorio con el contraste de Weber, $C_W = (L_{\text{texto}} - L_{\text{fondo}})/L_{\text{fondo}}$, con y sin la banda.

3.4.22 Síntesis de los prototipos

Los 19 prototipos confirman que las funciones críticas de EDDIE pueden reproducirse en Python con bibliotecas mantenidas. De ellos, 15 tienen resultados medidos (3 sobre el PDF real, 1 sobre el código final y 11 sintéticos) y 4 dependen por completo del hardware (F1, F9, F24 y F25); en los sintéticos, la parte que depende del sensor también queda marcada como pendiente. Además de confirmar la factibilidad, los prototipos entregaron cinco hallazgos que orientaron el diseño:

1. La binarización global de Otsu falla con la iluminación no uniforme del proyector (F2); en la versión final se aplica solo al recorte pequeño de la palabra resaltada.

2. El preprocesamiento empeora el OCR sobre texto limpio (F3); se justifica solo con imágenes de cámara degradadas.

3. El contenido proyectado del mismo color que el lápiz anula su seguimiento (F5); la interfaz evita esos colores (RNF-06).

4. Evaluar la compacidad antes que los vértices clasifica los cuadrados como círculos (F17); el plugin final sigue el orden del C#.

5. El clic por permanencia depende del radio, del tiempo y del ruido del sensor (F12); sus parámetros deben calibrarse con el eye tracker real.

CAPÍTULO 4. DISEÑO Y ASPECTOS DE IMPLEMENTACIÓN

Este capítulo corresponde a la etapa *Do* del ciclo PDCA y a los objetivos específicos 2 y 3. Describe la arquitectura del núcleo, los contratos de plugin, la capa de abstracción de hardware, los módulos integrados, el rediseño de la interfaz y las iteraciones en que se construyó el sistema.

4.1 ARQUITECTURA GENERAL

La nueva versión de EDDIE se organiza en cuatro capas (Figura 4.1). La **capa de contratos** (`contracts/`) define las interfaces abstractas que todo plugin debe implementar. El **núcleo** (`orchestrator/`) contiene el orquestador, el cargador de plugins, la fábrica de dispositivos y la traza de ejecución, y no depende de ningún módulo concreto. La **capa de plugins** (`hardware/` y `plugins/`) contiene las implementaciones: sensores reales y simulados, reconocimiento de texto, gestos, rastreo ocular, consistencia, búsqueda y voz. La **capa de presentación** (`app/` y `ui/`) contiene la interfaz de configuración, la pantalla proyectada y el puente `EddieRuntime` que las conecta con el orquestador.

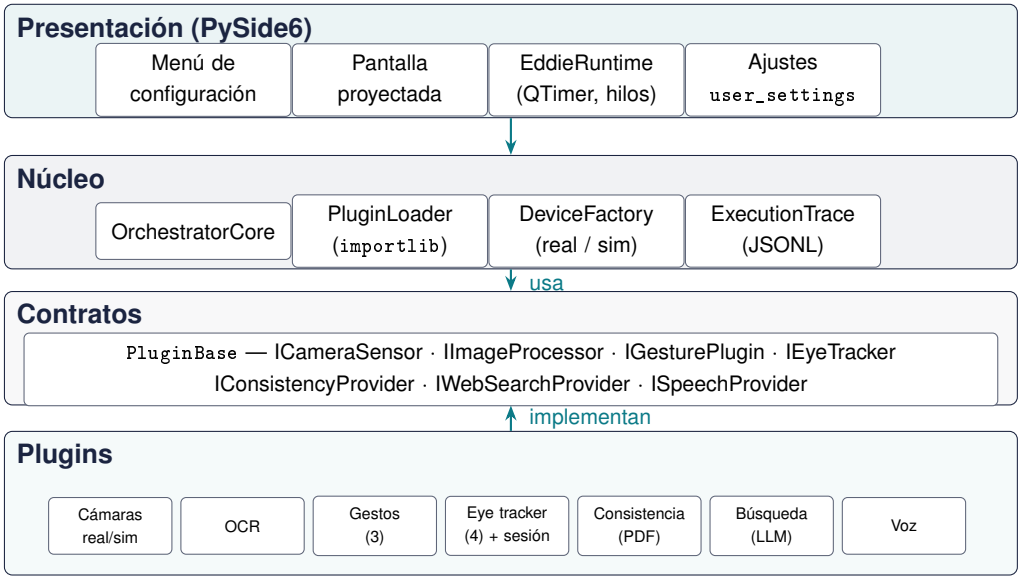


Figura 4.1: Arquitectura en capas de EDDIE en Python.

Fuente: Elaboración propia.

La dependencia va siempre hacia los contratos: el núcleo solo conoce `PluginBase` y los contratos de dominio, y los plugins solo conocen los contratos que implementan. Esto permite agregar un plugin sin modificar el núcleo (RNF-04) y reemplazar un dispositivo por su versión simulada sin modificar los plugins que lo consumen (RNF-05). A diferencia de EDDIE-2023 (Figura 3.1), existe un único camino de carga para todos los módulos.

4.2 CONTRATOS DE PLUGIN

Todo plugin hereda de `PluginBase`, que define tres métodos obligatorios, `initialize(config)`, `execute()` y `release()`, y la propiedad `name`. Sobre esa base, cada dominio tiene un contrato con las operaciones que el resto del sistema necesita (Tabla 4.1). Dos interfaces auxiliares resuelven el flujo de datos entre plugins: `ICameraFrameConsumer`, para los plugins que reciben un sensor de cámara inyectado, e `ICycleResultConsumer`, para los que necesitan los resultados de los demás plugins del mismo ciclo.

Tabla 4.1: Contratos de plugin y sus implementaciones.

Contrato	Operaciones principales	Implementaciones
`PluginBase`	`initialize`, `execute`, `release`, `name`	Todos los plugins
`ICameraSensor`	`capture_frame`, `get_latest_frame`	`CameraSensorPlugin`, `MockCameraSensor`
`IImageProcessor`	`preprocess`, `extract_text`, `recognize_highlighted_text`, `get_precision_recall`	`ImageProcessorPlugin`
`IGesturePlugin`	`detect_gesture`, `set_calibration`, `mark_selection_start/end`, `capture_selection`	`ColorPenPlugin`, `HandSkinPlugin`, `MousePlugin`
`IEyeTracker`	`get_gaze_point`, `is_connected`, `get_latency_ms`	`EyeTrackerPlugin` (GazePoint, TheEyeTribe, GazeCloud), `EyeTrackerMock`
`IConsistencyProvider`	`load_document`, `add_highlight`, `get_annotations`, `search_text`, marcadores y comentarios	`ConsistencyPlugin`
`IWebSearchProvider`	`search_encyclopedia`, `search_definition`, `translate_text`, `search_image`, `search_video`	`LLMSearchPlugin` (Gemini)
`ISpeechProvider`	`speak`, `stop_speaking`, `start_listening`, `stop_listening`	`SpeechPlugin`

Fuente: Elaboración propia a partir de `contracts/`.

4.3 NÚCLEO ORQUESTADOR

4.3.1 Ciclo de vida

El `OrchestratorCore` lee `config/plugins.json`, superpone los ajustes del usuario, carga los plugins con el `PluginLoader` y los inicializa respetando sus dependencias (por ejemplo, el reconocimiento de texto depende de la cámara de texto). En cada ciclo llama a `execute()` de cada plugin, primero los productores y luego los consumidores de resultados, y mide la latencia de cada uno. Al cerrar, libera los plugins en orden inverso al de inicialización. El Algoritmo 1 resume el ciclo; el aislamiento de fallos está en las líneas marcadas con `try`.

Algoritmo 1: Ciclo de vida del orquestador.

Entrada: archivo `plugins.json`, ajustes del usuario
Salida : resultados por ciclo, métricas de latencia, traza

$C \leftarrow$ leer configuración y superponer ajustes;
para cada *entrada* $e \in C$ *habilitada* **hacer**
    $clase \leftarrow$ `DeviceFactory.resolve`(e) ; `// real o simulada`
    try cargar $clase$ con `importlib` **except** registrar `load_failed`;
para cada *plugin* p *cargado, en orden de dependencias* **hacer**
    inyectar sensor declarado si p es `ICameraFrameConsumer`;
    try p.`initialize`$(config_p)$ **except** registrar falla y continuar;
mientras *el sistema esté activo* **hacer**
    $R \leftarrow \emptyset$;
    para cada *plugin activo* p *(productores primero, consumidores al final)* **hacer**
        $t_0 \leftarrow$ `perf_counter`();
        si p *es* `ICycleResultConsumer` **entonces**
            p.`consume_cycle_results`(R);
        try $R[p] \leftarrow p$.`execute`$()$ **except** $R[p] \leftarrow$ error, registrar;
        registrar latencia $t - t_0$; advertir si supera 8,3 ms;
para cada *plugin* p *en orden inverso* **hacer**
    try p.`release`$()$ **except** registrar;

Además del ciclo, el orquestador ofrece tres operaciones que necesita la interfaz: `invoke(plugin, método, ...)`, que ejecuta una operación bajo demanda (por ejemplo, reconocer el texto resaltado o buscar una definición) con el mismo aislamiento de fallos y midiendo su latencia; `restart_plugin`, que reinicia un plugin con una configuración nueva y reconecta a sus consumidores; y `stop_plugin`. Cada acción queda registrada en una traza JSONL que sirve de evidencia para la evaluación.

4.3.2 Configuración declarativa

Todos los plugins se declaran en un único archivo. Cada entrada indica la clase, la implementación simulada si es un dispositivo, las dependencias y la configuración:

```
{ "name": "ImageProcessor",
  "class": "plugins.image_processor_plugin.ImageProcessorPlugin",
```

```
"enabled": true,
"dependencies": {"camera_sensor": "TextCamera"},
"config": {"ocr_language": "spa+eng", "rotate_180": true} }
```

Agregar un módulo consiste en escribir la clase y agregar una entrada; si la clase no existe o tiene un error, el plugin queda registrado como fallido y el resto del sistema arranca igual, que es exactamente el caso que en EDDIE-2023 detenía la aplicación (E50). Los ajustes del usuario (cámara elegida, PDF, eye tracker) se guardan en un archivo aparte, `config/user_settings.json`, con las mismas claves que usaba `Settings2` en C#, y se superponen al cargar sin modificar `plugins.json`.

4.3.3 Capa de abstracción de hardware

La `DeviceFactory` decide, para cada dispositivo que declara `simulated_class`, si se carga la implementación real o la simulada. El modo se elige con la variable de entorno `EDDIE_DEVICE_MODE` (`real` o `sim`) o por dispositivo en su configuración. Las implementaciones simuladas generan señales con verdad terreno conocida: `MockCameraSensor` entrega una página con texto resaltado, un lápiz o una mano en posiciones conocidas, y `EyeTrackerMock` una trayectoria de mirada. Esto permite ejecutar el sistema completo y las pruebas automatizadas sin el laboratorio (RNF-05) y, en una sesión real, distinguir una falla del sensor de una falla del software: si el sistema funciona en modo simulado y no en el real, el problema está en el dispositivo o su conexión.

4.3.4 Integración con la interfaz

`EddieRuntime` conecta el orquestador con la interfaz. Un temporizador de Qt ejecuta `execute_cycle()` a 30 Hz en el hilo de la interfaz, mientras que los sensores capturan en hilos propios, de modo que un ciclo nunca espera a la cámara. Las acciones lentas (OCR, consulta al modelo de lenguaje, guardado del PDF, voz) se ejecutan con `invoke` en un conjunto de hilos de trabajo y su resultado vuelve a la interfaz por una señal. Este esquema reemplaza los hilos y ventanas de carga *ad hoc* de EDDIE-2023.

4.4 MÓDULOS INTEGRADOS

Reconocimiento de texto. `ImageProcessorPlugin` recibe la cámara de texto, localiza la zona resaltada, la recorta y la transcribe con Tesseract (`spa+eng`). También

calcula precisión y recall contra una lista de palabras esperadas, la métrica de la propuesta para la verificación funcional.

Interacción gestual. Tres plugins implementan `IGesturePlugin`: lápiz de color, mano y mouse. Comparten la calibración de cuatro puntos, la selección de un fragmento y su captura, y una opción común para rotar la imagen 180° cuando la cámara está montada invertida (hallazgo H-01).

Rastreo ocular. `EyeTrackerPlugin` implementa los protocolos de GazePoint (Open Gaze API, TCP 4242), TheEyeTribe (TCP 6555) y GazeCloud (WebSocket 3000). `GazeSessionPlugin` registra la sesión de mirada en CSV con las columnas de EDDIE-2023 y el clic por permanencia.

Consistencia. `ConsistencyPlugin` mantiene el PDF abierto y sincroniza resaltados, comentarios, marcadores y figuras en ambos sentidos con PyMuPDF. Cada guardado escribe una copia completa y reemplaza el original (hallazgo H-02).

Búsqueda. `LLMSearchPlugin` implementa los cinco servicios de `IWebSearchProvider` con Gemini 2.5 Flash. La clave se lee de `config/secrets.json` o de la variable `GEMINI_API_KEY`, que no se versionan.

Voz. `SpeechPlugin` lee los resultados con `pyttsx3` y escucha los comandos con `SpeechRecognition`.

4.5 DISEÑO DE LA INTERFAZ

La interfaz de EDDIE-2023 se reconstruyó primero como réplica de los formularios WinForms (a partir de los archivos `.Designer.cs`) para no perder ningún control, y luego se rediseñó conservando todos los controles y flujos con el mismo nombre y comportamiento (RNF-06). Los criterios del rediseño provienen de la evaluación de Ibaceta Jaña (2023) y de la operación en la sala:

- **Sin amarillo fuera del resaltado.** La cámara detectaba el botón de resaltado de la proyección en lugar del texto (EFD2). La paleta usa tinta azul oscura (#1B2440) y un acento verde azulado (#0B7A87); el amarillo queda reservado para el resaltado, que es lo que el OCR busca.
- **Controles de la proyección en la franja superior.** TheEyeTribe no detectaba la parte inferior de la proyección (E49) y el reconocimiento de texto fallaba lejos de

la cámara (EFD4), por lo que los botones que se activan con la mirada se ubican en la franja superior. La zona de lectura y las cruces de calibración conservan su posición original, porque los algoritmos de reconocimiento dependen de ellas.

- **Estado real de cada módulo.** La pantalla de inicio muestra los pasos para preparar la sesión y el estado de cada módulo según el orquestador (U1), y los mensajes son avisos no modales que no detienen la lectura (U3).
- **Calibración desde el sensor.** Los puntos de calibración gestual se copian de la lectura actual del sensor en lugar de escribirse a mano (U2).
- **Español en toda la interfaz** (E14).

Las Figuras 4.2 a 4.5 muestran la interfaz final; la comparación con EDDIE-2023 está en el Apéndice C.



Figura 4.2: Pantalla de inicio con los pasos de preparación y el estado de cada módulo.

Fuente: Elaboración propia (captura del avance P-07 con dispositivos simulados).

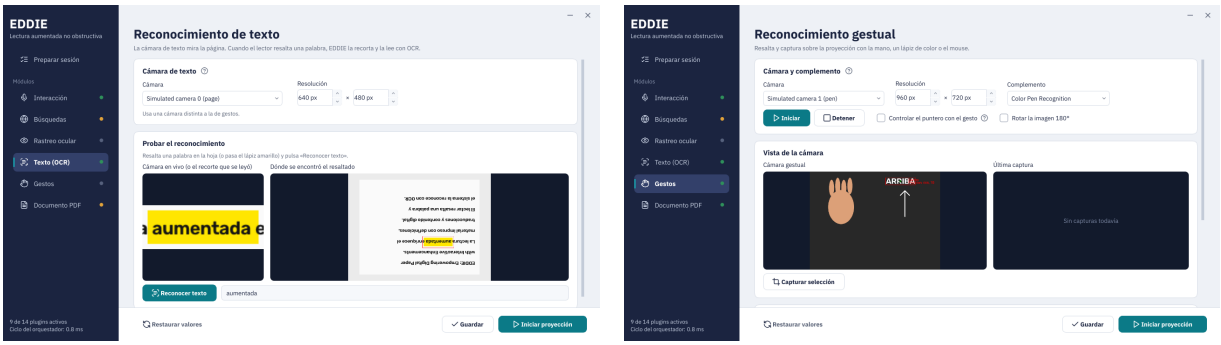


Figura 4.3: Configuración del reconocimiento de texto (izquierda) y de la interacción gestual con lápiz (derecha).

Fuente: Elaboración propia (capturas del avance P-07).

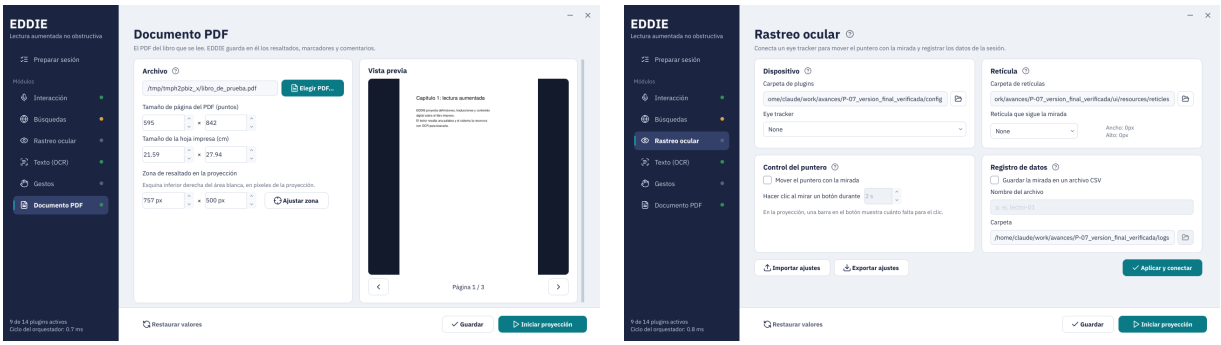


Figura 4.4: Configuración del documento PDF (izquierda) y del rastreo ocular (derecha).

Fuente: Elaboración propia (capturas del avance P-07).

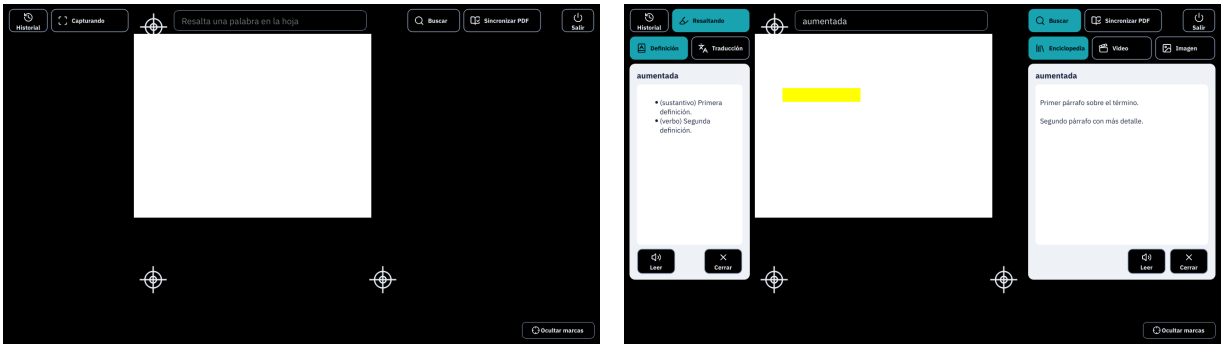


Figura 4.5: Pantalla proyectada: controles en la franja superior (izquierda) y resultado de una búsqueda (derecha).

Fuente: Elaboración propia (capturas del avance P-07 con un modelo de lenguaje de prueba).

4.6 ITERACIONES DE DESARROLLO

El sistema se construyó en ocho avances, cada uno con su código completo, pruebas y evidencias reproducibles (Tabla 4.2). P-00 corresponde a los prototipos desechables del Capítulo 3; P-01 a P-06 integran el núcleo y los módulos en el orden de menor a mayor dependencia del hardware; P-07 es la versión verificada. La interfaz se incorpora desde P-03 y muestra, en cada avance, solo los módulos ya integrados: los demás aparecen deshabilitados con la etiqueta del avance en que se integran. Las fichas de cada avance, con sus objetivos y requisitos, están en el Apéndice A.

Tabla 4.2: Avances del desarrollo y resultados de sus pruebas.

Avance	Contenido	PDCA	Pruebas	Fallos esperados	Cobertura
P-00	Prototipos desechables y mediciones	Plan	—	—	—
P-01	Núcleo orquestador y contratos	Do	34	0	81 %
P-02	Capa de hardware y consistencia	Do	41	1	73 %
P-03	Reconocimiento de texto e interfaz	Do	65	0	67 %
P-04	Interacción gestual	Do	79	1	72 %
P-05	Rastreo ocular	Do	94	1	74 %
P-06	Proyección, búsqueda con LLM y voz	Do	100	2	81 %
P-07	Versión final verificada	Check/Act	107	0	81 %

Fuente: Elaboración propia a partir de `evidencias/` de cada avance.

Los fallos esperados (*xfail*) son pruebas que documentan un defecto conocido: en los avances en que el defecto existe la prueba se marca como fallo esperado, y en P-07, con la corrección aplicada, pasa a aprobada. Así cada corrección queda respaldada por una prueba que demuestra el problema y su solución (Sección 5.2.5).

CAPÍTULO 5. EVALUACIÓN

Este capítulo corresponde a la etapa *Check* del ciclo PDCA y al cuarto objetivo específico. La evaluación se organiza en los tres niveles del diseño experimental de la propuesta: pruebas de software, verificación y validación con MECIVA PS, y pruebas de aceptación en el laboratorio. El primer nivel se ejecutó completo con dispositivos simulados; los dos siguientes requieren el hardware del laboratorio, y para ellos se presentan el protocolo, los instrumentos ya construidos y los resultados que faltan, marcados como [Pendiente: laboratorio].

5.1 PLAN DE EVALUACIÓN

La Tabla 5.1 resume las métricas comprometidas, su criterio de aceptación y su estado. Todas las métricas automatizadas se regeneran con `python tools/generar_evidencias.py` en cada avance, y los resultados de este capítulo provienen de la carpeta `evidencias/` del avance P-07 (Linux 6.18.44-fc-v310, Python 3.13.18, 2026-11-08).

Tabla 5.1: Métricas de evaluación, criterio de aceptación y estado.

Métrica	Criterio	Estado
Nivel 1: pruebas de software		
Cobertura de líneas	$\geq$ 80 % (RNF-03)	81 %, cumple
Latencia orquestador $\rightarrow$ plugin	p95 $\leq$ 8,3 ms (RNF-01)	1,48 ms el ciclo completo, cumple
Aislamiento de fallos	Una falla no detiene el sistema (RNF-02)	Verificado por pruebas
Matriz de trazabilidad	Cada caso: corregido, persistente o fuera de alcance	Parcial (Sección 5.2.4)
Desfase entre sensores y pérdida de eventos	Desfase máximo (ms) y tasa de pérdida en captura simultánea	[Pendiente: laboratorio]
Precisión y recall del OCR	$\geq$ 10 palabras de distinto largo	Métrica implementada; [Pendiente: cámara]
Tasa de error gestual	Gestos bien clasificados / ejecutados	[Pendiente: laboratorio]
Nivel 2: verificación y validación (MECIVA PS)		
Requisitos críticos	100 % de las ejecuciones	[Pendiente: laboratorio]
Validez funcional	% de requisitos heredados cumplidos	[Pendiente: laboratorio]
Confiabilidad	CV o tasa de éxito, $\geq$ 10 ejecuciones por caso	[Pendiente: laboratorio]
Nivel 3: pruebas de aceptación		
Requisitos secundarios	Escala A–E (Tabla 5.5)	[Pendiente: laboratorio]

Fuente: Elaboración propia a partir de la propuesta de tesis.

5.2 PRUEBAS DE SOFTWARE

5.2.1 Pruebas automatizadas y cobertura

La versión final tiene 107 pruebas automatizadas con pytest, organizadas en cinco archivos: núcleo y cargador (`test_orchestrator.py`), hardware y núcleo extendido

(`test_hardware_and_core.py`), contratos y plugins (`test_plugins_eddie.py`), flujos de la interfaz (`test_ui_flows.py`) e interfaz rediseñada (`test_interfaz.py`). Las pruebas de interfaz ejecutan los flujos reales de la aplicación (abrir un PDF, sincronizar un resaltado, calibrar un gesto, abrir la proyección) con dispositivos simulados y sin pantalla. Todas aprueban y la cobertura de líneas es de 81 %, sobre el umbral de 80 % comprometido.

La Tabla 4.2 (Capítulo 4) muestra la evolución por avance. La cobertura de P-02 a P-05 está bajo el umbral porque esos avances contienen código de interfaz o de dispositivos que todavía no tenía pruebas de flujo completas; el umbral se exige a la versión final, que es la que se entrega.

5.2.2 Latencia del orquestador

Se midieron 500 ciclos completos del orquestador, tras 10 de calentamiento, con todos los plugins activos y el reconocimiento con lápiz encendido (Tabla 5.2). El ciclo completo, que incluye a todos los plugins, tiene una latencia de 0,59 ms en promedio y 1,48 ms en el percentil 95, con un máximo de 2,31 ms. Ningún plugin superó los 8,3 ms en ningún ciclo, por lo que RNF-01 se cumple con margen: el ciclo completo usa menos del 10 % del período entre dos muestras del eye tracker.

Tabla 5.2: Latencia por plugin en 500 ciclos del orquestador (ms).

Plugin	**Media**	**p95**	**Máx.**
TextCamera	0,117	0,182	0,76
GestureCamera	0,264	0,369	2,12
ImageProcessor	0,006	0,008	0,057
ColorPenRecognition	0,122	0,932	1,189
HandSkinRecognition	0,003	0,005	0,046
MouseRecognition	0,002	0,003	0,013
EyeTrackerMock	0,01	0,015	0,16
SpeechProvider	0,003	0,004	0,079
GazeSession	0,005	0,006	0,13
ConsistencyProvider	0,009	0,011	0,074
Ciclo completo	0,59	1,48	2,31

Fuente: Elaboración propia, `evidencias/latencias.json` de P-07 (dispositivos simulados).

Dos limitaciones acotan este resultado. Primero, se midió con dispositivos simu-

lados: la latencia de captura de la cámara real y la del eye tracker ocurren en hilos propios y no forman parte del ciclo, pero el costo del reconocimiento sobre imágenes reales puede ser mayor. Segundo, en P-06 se registró un ciclo aislado de 10,28 ms con un percentil 95 de 1,49 ms; corresponde a una pausa del sistema operativo o del recolector de memoria y no se repitió en P-07, pero muestra que el máximo debe medirse también en el PC del laboratorio con `generar_evidencias.py -real`.

5.2.3 Aislamiento de fallos

El aislamiento de fallos (RNF-02) se verifica con pruebas que provocan fallas en cada fase del ciclo de vida: una clase que no existe o un módulo inválido (`test_load_invalid_module`, `test_load_all_missing_class_key`), un archivo de configuración ausente (`test_initialize_with_missing_config_file`), un plugin cuya inicialización falla (`test_initialize_plugin_init_fail_does_not_stop_system`), una excepción en `execute()` (`test_execute_plugin_exception_isolation`), una excepción al liberar (`test_release_exception_does_not_stop_others`) y una operación bajo demanda que falla (`test_invoke_isolates_failures`). En todos los casos el resto de los plugins sigue funcionando y la falla queda registrada con su causa.

El informe de inicialización de P-07 muestra el mismo comportamiento en la aplicación real: sin clave de Gemini configurada, el plugin de búsqueda queda como `initialization_failed` y los otros trece plugins se inicializan normalmente. En EDDIE-2023, una situación equivalente (una DLL o carpeta de plugins faltante, E50) impedía usar el módulo completo.

5.2.4 Matriz de trazabilidad

La propuesta compromete reejecutar los casos de prueba documentados por Pezoa Riquelme (2020), Contreras Velásquez (2019) y Estay Vera (2020) en EDDIE-2023 y en la versión en Python, y registrar cada error de Ibaceta Jaña (2023) como corregido, persistente o fuera de alcance, con su evidencia. La Tabla 5.3 presenta la parte de la matriz que puede respaldarse con pruebas automatizadas; los casos que dependen del hardware quedan como pendientes hasta su ejecución en el laboratorio. La matriz completa de los 67 errores se construye con la plantilla del Anexo A.

Tabla 5.3: Matriz de trazabilidad (parcial) de errores de EDDIE-2023.

Error	Descripción	Cat.	Resultado	Evidencia
E50	Carpeta `PluginsConsistencia` no encontrada	(a)	Corregido por diseño	`test_load_invalid_module`, `test_load_all_missing_class_key`
E7, E8	No inicia sin cámara o sin dos cámaras	(d)	Corregido	`test_camera_sensor_fails_cleanly_when_camera_missing`; modo simulado
E11, E21, E22, E30	Fallas al sincronizar, buscar o revisar sin PDF o sin página	(d)	Corregido	`test_missing_pdf_is_reported_not_crashing`, `test_pdf_tools_require_pdf_and_page_number`
E12, E13	Sin manual de instalación ni de uso	(c)	Corregido	README del repositorio y manual del Apéndice D
E14	Mensajes en otro idioma	(e)	Corregido	Interfaz y mensajes en español (RNF-06)
EFD2	La cámara detecta botones de la interfaz	(e)	Corregido por diseño; [Pendiente: verificar con cámara]	`test_interface_chrome_never_uses_yellow`
E27, E28	Código sin comentarios ni nombres representativos	(d)	[Pendiente: SonarQube]	Docstrings y análisis estático
E40, E41	Resaltado digital no coincide con el físico	(d)	[Pendiente: laboratorio]	Prueba de aceptación de consistencia
E15, E16	Eye tracker no reconocido; cámara inestable	(b)	[Pendiente: laboratorio]	Modo real vs. simulado (DeviceFactory)
E49	No se detecta la parte inferior de la proyección	(b)	Mitigado por diseño; [Pendiente: laboratorio]	Controles en la franja superior; prueba con GazePoint
APIs de búsqueda	Credenciales vencidas	(g)	Corregido por reemplazo	`test_gemini_key_from_secrets_file_or_missing` y servidor de prueba
E9, E10	Conexiones eléctricas y seguridad del equipo	(f)	Fuera de alcance	Infraestructura del laboratorio

Fuente: Elaboración propia a partir de Ibaceta Jaña (2023) y de las pruebas de P-07. Categorías de la Tabla 3.1: (a) carga y despliegue, (b) hardware, (c) documentación, (d) práctica de código, (e) interfaz e idioma, (f) infraestructura, (g) servicios externos.

5.2.5 Hallazgos de la verificación

Durante el desarrollo y la verificación se encontraron tres defectos que las pruebas automatizadas no habían detectado. Cada uno se documentó con una prueba que lo reproduce y se corrigió (Tabla 5.4).

Tabla 5.4: Hallazgos de la verificación y su corrección.

ID	Problema	Corrección y prueba
H-01	Al calibrar con la mano la imagen aparecía rotada 180°: el plugin heredó de `HandSkinRecognition.cs` una rotación fija pensada para la cámara invertida del laboratorio.	La rotación pasa a ser una opción común de los tres plugins gestuales, desactivada por defecto, con una casilla en la interfaz. Pruebas `test_gesture_preview_keeps_camera_orientation_by_default` y relacionadas.
H-02	Con PyMuPDF 2.1, desde el segundo guardado incremental el PDF quedaba con su tabla de referencias dañada y al reabrirlo se perdían anotaciones.	Cada guardado escribe una copia completa y reemplaza el original, con reintento si el archivo está bloqueado. Pruebas `test_successive_saves_keep_every_annotation_on_disk` y relacionadas.
H-03	Los botones de ayuda de Rastreo ocular lanzaban una excepción en cada clic porque la señal de Qt reemplazaba el argumento del *callback*.	La función ignora los argumentos de la señal. Prueba `test_info_icons_and_buttons_with_help_do_not_raise`.

Fuente: Elaboración propia.

Los tres hallazgos tienen un rasgo común: el programa parecía funcionar. En H-02 el documento en memoria era correcto y el daño solo aparecía al reabrir el archivo; en H-03 la acción del botón se ejecutaba y la excepción quedaba en el registro. Esto refuerza la necesidad de las pruebas de aceptación con el hardware real, porque hay defectos que solo se observan al operar el sistema completo.

5.3 VERIFICACIÓN Y VALIDACIÓN CON MECIVA PS

5.3.1 Protocolo

La verificación replica la fase de diagnóstico de MECIVA PS (Ibaceta Jaña, 2023) para cada módulo del MVP: consistencia, reconocimiento de texto, interacción gestual y rastreo ocular. Para cada módulo:

1. Se clasifican sus requisitos en **críticos**, que deben cumplirse en el 100 % de las ejecuciones y cuyo incumplimiento es bloqueante (por ejemplo, que los plugins del MVP carguen sin fallas, o que el error espacial del rastreo ocular esté dentro del margen definido), y **secundarios**, que se evalúan con la escala de la Tabla 5.5.

2. Se ejecuta cada caso al menos 10 veces con el hardware real (`EDDIE_DEVICE_MODE=real`) y se registra el resultado en la plantilla de diagnóstico de MECIVA PS (funcionalidad, resultado esperado, método, pruebas, resultados y nivel de funcionamiento).

3. Se calcula la **validez funcional** como el porcentaje de requisitos heredados que el módulo cumple, y la **confiabilidad** como el coeficiente de variación $\mathrm{CV} = \sigma/\mu$ para variables continuas (por ejemplo, las coordenadas de una fijación sobre un punto fijo) o como la tasa de éxito para resultados binarios (por ejemplo, que un comentario se sincronice).

Para el reconocimiento de texto se usa el mismo conjunto de Ibaceta Jaña (2023): diez palabras de distinto largo (Control, Arte, Lectura, Jerusalén, Esponja, Pastilla, Caramelo, Metodología, Consistencia y Cama) en nueve posiciones de la zona de lectura, con diez repeticiones por posición. Así los resultados son directamente comparables con la línea base de 2023, en que la validez varió entre 10 % y 100 % según la posición. El plugin calcula precisión y recall con `get_precision_recall` (verificado en `test_precision_recall_metric`):

$$\mathrm{Precisin} = \frac{|P_{\text{reconocidas}} \cap P_{\text{esperadas}}|}{|P_{\text{reconocidas}}|}, \qquad \mathrm{Recall} = \frac{|P_{\text{reconocidas}} \cap P_{\text{esperadas}}|}{|P_{\text{esperadas}}|}. \tag{5.1}$$

5.3.2 Resultados

[Pendiente: Tabla de validez y confiabilidad por módulo, con la línea base de Ibaceta Jaña (2023) en una columna para comparar.]

5.4 PRUEBAS DE ACEPTACIÓN

Las pruebas de aceptación se realizan en la sala del laboratorio InTeractiOn con el profesor guía como evaluador, siguiendo la práctica de los trabajos anteriores del laboratorio (Estay Vera, 2020; Pezoa Riquelme, 2020; Figueroa Contreras, 2025). Cada prueba tiene precondiciones, pasos, requisitos abordados y resultado esperado, y se califica con la escala de la Tabla 5.5. Los escenarios cubren la operación simultánea de módulos, la recuperación ante la falla parcial de un sensor (desconectar una cámara durante la sesión) y la sincronización de datos entre componentes de frecuencias distintas.

Tabla 5.5: Escala de evaluación de las pruebas de aceptación.

Evaluación	Significado	Cumplimiento
A	Completa	85–100 %
B	Aceptada con observaciones	55–84,9 %
C	Incompleta	45–54,9 %
D	Insuficiente	15–44,9 %
E	No aceptada	0–14,9 %

Fuente: Estay Vera (2020).

Tabla 5.6: Pruebas de aceptación planificadas.

ID	Prueba	Requisitos	Resultado
PA-01	Arrancar con todos los plugins del MVP, desconectar una cámara durante la sesión y reconectarla desde la interfaz.	N1, N2, N6, RNF-02	[Pendiente:]
PA-02	Resaltar una palabra en el papel y obtener su texto, su definición y su traducción.	FD1, B1–B13	[Pendiente:]
PA-03	Sincronizar resaltado, comentario, marcador y figura del papel al PDF y del PDF al papel.	C1–C12	[Pendiente:]
PA-04	Calibrar el lápiz y la mano, seleccionar un fragmento y capturarlo.	G1–G3, U2	[Pendiente:]
PA-05	Conectar GazePoint y TheEyeTribe, ver la retícula y activar un botón de la proyección con la mirada.	RO1–RO7	[Pendiente:]
PA-06	Registrar una sesión de lectura y verificar el CSV de mirada y la traza del orquestador.	RO4, N7	[Pendiente:]
PA-07	Preparar una sesión completa desde la pantalla de inicio siguiendo el manual.	U1, U3, RFS1–RFS8	[Pendiente:]

Fuente: Elaboración propia.

5.5 SÍNTESIS DE LA EVALUACIÓN

Las pruebas de software muestran que la arquitectura cumple los requisitos no funcionales que pueden verificarse sin el laboratorio: cobertura de 81 % (RNF-03), latencia del ciclo completo de 1,48 ms en el percentil 95 (RNF-01), aislamiento de fallos en todas las fases del ciclo de vida (RNF-02), extensibilidad por configuración (RNF-04) y ejecución completa con dispositivos simulados (RNF-05). De los errores de EDDIE-2023, los de carga y despliegue, práctica de código, documentación e idioma que se pudieron reproducir quedan corregidos y respaldados por pruebas. Los

errores de hardware y la coincidencia física del resaltado (E40, E41, E49) solo pueden cerrarse con las pruebas del laboratorio, para las que el protocolo y los instrumentos de medición ya están disponibles.

CAPÍTULO 6. CONCLUSIONES

Versión preliminar: las conclusiones sobre el cuarto objetivo se completan con los resultados de la verificación funcional y de las pruebas de aceptación en el laboratorio.

6.1 CUMPLIMIENTO DE LOS OBJETIVOS

OE1: diagnóstico y factibilidad. La reclasificación por causa de los 67 errores de Ibaceta Jaña (2023) mostró que 42 de ellos (62,7 %) dependen de la arquitectura (carga y despliegue, y práctica de código) y no del hardware, lo que justifica reconstruir el núcleo en lugar de corregir errores aislados. Los 19 prototipos desechables confirmaron que las funciones críticas pueden reproducirse en Python con bibliotecas mantenidas y, al medirse con verdad terreno conocida, entregaron cinco hallazgos que no eran visibles en el código del sistema legado: la binarización global falla con la luz del proyector, el preprocesamiento empeora el OCR sobre texto limpio, el contenido proyectado del color del lápiz anula su seguimiento, el orden de la regla de clasificación de figuras confunde cuadrados con círculos y el clic por permanencia depende fuertemente del ruido del sensor. El objetivo se considera cumplido, con las mediciones que dependen del hardware (F1, F9, F24, F25 y la parte de sensor de las demás) pendientes de ejecutar en el laboratorio con el mismo script.

OE2: diseño del núcleo. Se definió un contrato base único (`PluginBase`) con ciclo de inicialización, ejecución y liberación, siete contratos de dominio y dos interfaces de flujo de datos, un único archivo de configuración y una capa de abstracción de hardware con conmutación real/simulado. El diseño reemplaza los tres mecanismos de carga de EDDIE-2023 por uno solo. El objetivo se considera cumplido.

OE3: implementación e integración. Se implementó el núcleo y se integraron, en siete iteraciones, los módulos de consistencia, reconocimiento de texto, interacción gestual (lápiz, mano y mouse), rastreo ocular (GazePoint, TheEyeTribe, GazeCloud y simulado), búsqueda con modelo de lenguaje y voz, junto con la interfaz de configuración y la pantalla proyectada. La interfaz conserva todos los controles y flujos de EDDIE-2023 y corrige los problemas de usabilidad registrados en 2023. El objetivo se

considera cumplido en software; la integración con el hardware físico se confirma en las pruebas de aceptación.

OE4: validación. Las pruebas de software se completaron: 107 pruebas aprobadas, 81 % de cobertura de líneas, latencia del ciclo completo de 1,48 ms en el percentil 95 frente al umbral de 8,3 ms y aislamiento de fallos verificado en todas las fases del ciclo de vida. La verificación funcional con MECIVA PS y las pruebas de aceptación [Pendiente: se ejecutan en el laboratorio]; sus protocolos e instrumentos están listos (Capítulo 5).

6.2 RESPUESTA A LA PREGUNTA DE INVESTIGACIÓN

La pregunta planteada fue en qué medida la reconstrucción del núcleo bajo una arquitectura de plugins con contrato único permite reducir los errores estructurales documentados y habilitar el uso de EDDIE como instrumento de medición confiable. Respecto de la primera parte, los errores de carga y despliegue dejan de ser posibles por construcción: un plugin faltante o defectuoso se registra y el resto del sistema arranca, y los errores de práctica de código que se reprodujeron quedan respaldados por pruebas. Respecto de la segunda, la arquitectura entrega las condiciones necesarias para medir (latencia acotada, traza de cada acción, ejecución reproducible con dispositivos simulados), pero la confiabilidad de las mediciones con el hardware real solo puede afirmarse después de la verificación con MECIVA PS.

6.3 LIMITACIONES

- Los resultados de los prototipos marcados como sintéticos miden el algoritmo sobre datos con verdad terreno conocida, no el desempeño con la cámara o el eye tracker reales.
- Las pruebas de software se ejecutaron con dispositivos simulados; la latencia y el comportamiento con el hardware real deben medirse en el PC del laboratorio.
- La búsqueda depende de un servicio externo (Gemini) y de una clave personal; si el servicio cambia sus condiciones, el contrato `IWebSearchProvider` permite reemplazarlo, pero el reemplazo no está implementado.
- El desarrollo y la validación se realizan solo en Windows 11.

6.4 TRABAJO FUTURO

- Ejecutar la verificación con MECIVA PS y las pruebas de aceptación, y completar la matriz de trazabilidad de los 67 errores.
- Calibrar los parámetros del clic por permanencia y los umbrales de color con el hardware del laboratorio.
- Implementar un proveedor de búsqueda local (por ejemplo, un modelo de lenguaje ejecutado en el equipo) para no depender de un servicio externo.
- Retomar la detección automática del libro (F4), que resultó factible, y el canal de telemetría para el visor web que quedó fuera del alcance.

GLOSARIO

Arquitectura de plugins: organización de un sistema en un núcleo estable y módulos intercambiables que implementan un contrato común y se pueden agregar o reemplazar sin modificar el núcleo.

Capa de abstracción de hardware (HAL): componente que oculta los detalles de cada dispositivo detrás de una interfaz uniforme, de modo que un dispositivo real puede reemplazarse por uno simulado.

Clic por permanencia (*dwell*): activación de un control cuando la mirada permanece dentro de un radio durante un tiempo mínimo.

Consistencia bidireccional: en EDDIE, mantener equivalentes el documento impreso y su versión PDF, de modo que una marca hecha en uno se refleje en el otro.

Contrato de interfaz: conjunto de operaciones que un módulo se compromete a implementar; en este trabajo, una clase abstracta de Python.

Deuda técnica: costo futuro de retrabajo derivado de elegir soluciones rápidas en lugar de un diseño mantenible.

Eye tracker: dispositivo que estima, a una frecuencia fija, el punto al que mira una persona.

Fijación: período en que la mirada permanece estable sobre una región.

Homografía: transformación proyectiva entre dos planos, usada para convertir coordenadas de la cámara en coordenadas de la proyección.

Interacción humano-información (HII): campo que estudia cómo las personas buscan, evalúan y usan información en distintos formatos.

Latencia: tiempo entre una acción y su resultado; en este trabajo, el tiempo que el orquestador tarda en ejecutar un plugin o un ciclo completo.

MECIVA PS: metodología para la evaluación y validación de productos de software desarrollados para EDDIE (Ibaceta Jaña, 2023).

Modelo de lenguaje (LLM): modelo de aprendizaje automático que genera texto a partir de una instrucción; en EDDIE responde los servicios de búsqueda.

OCR: reconocimiento óptico de caracteres; conversión de la imagen de un texto en caracteres.

Orquestador: componente central que controla el ciclo de vida de los plugins y coordina el flujo de datos entre ellos.

Percentil 95 (p95): valor bajo el cual queda el 95 % de las mediciones.

Prototipo desechable: pieza de software acotada que se construye para evaluar la factibilidad de una función y luego se descarta.

QuadPoints: coordenadas de los cuatro vértices de cada fragmento de texto en un PDF, usadas para resaltarlo.

Realidad aumentada espacial: superposición de información digital directamente sobre los objetos físicos mediante un proyector.

***Science gateway*:** plataforma que integra instrumentos, datos y flujos de trabajo científicos.

Software científico: software desarrollado *ad hoc* para soportar una tarea de investigación.

SonarQube: plataforma de análisis estático de la calidad, la seguridad y la mantenibilidad del código.

Toque de Midas: activación involuntaria de controles por el solo hecho de mirarlos.

REFERENCIAS BIBLIOGRÁFICAS

Bimber, O. and Raskar, R. (2005). *Spatial Augmented Reality: Merging Real and Virtual Worlds*. A K Peters.

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.

Bustio Martínez, L., Coma Peña, Y., and Talavera Bustamante, I. (2013). Arquitectura basada en plugins para el desarrollo de software científico. In *XV Convención y Feria Internacional Informática*, La Habana, Cuba.

Carrasco Madriaga, O. I. (2017). Sistema para el estudio de interacciones con material impreso y digital equivalentes a través de realidad aumentada. Trabajo de titulación, Universidad de Santiago de Chile.

Contreras Velásquez, C. E. (2019). Desarrollo de un sistema para soportar lectura aumentada no obstrusiva. Trabajo de titulación, Universidad de Santiago de Chile.

Ernst, N. A., Bavota, G., and Cai, Y. (2026). The nature of technical debt in research software. Preprint, arXiv. `https://arxiv.org/abs/2603.2041`. El identificador del borrador (2603.2041) no tiene el formato de arXiv (AAMM.NNNNN): verificarlo.

Estay Vera, G. H. (2020). Módulo de interacciones basadas en movimientos oculares para ambiente de lectura aumentada no obstrusiva. Trabajo de titulación, Universidad de Santiago de Chile.

Figueroa Contreras, M. J. (2025). Actualización del núcleo de multimodal observer (MO). Tesis de ingeniería civil en informática, Universidad de Santiago de Chile.

Glatard, T., Rousseau, M. E., Camarasu-Pop, S., Adalat, R., Beck, N., Das, S., Ferreira da Silva, R., Khalili-Mahani, N., Korkhov, V., Quirion, P. O., et al. (2017). Software architectures to integrate workflow engines in science gateways. *Future Generation Computer Systems*, 75:239–255.

Hartley, R. and Zisserman, A. (2004). *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition.

Ibaceta Jaña, J. M. (2023). Metodología para la evaluación y validación de productos de software desarrollados para EDDIE. Tesis de ingeniería civil en informática, Universidad de Santiago de Chile.

Jacob, R. J. K. (1990). What you look at is what you get: Eye movement-based interaction techniques. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90)*, pages 11–18.

Koike, H., Sato, Y., and Kobayashi, Y. (2001). Integrating paper and digital information on EnhancedDesk: A method for real-time finger tracking on an augmented desk system. *ACM Transactions on Computer-Human Interaction*, 8(4):307–322.

Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710.

Megargel, A., Poskitt, C. M., and Shankararaman, V. (2021). Microservices orchestration vs. choreography: A decision framework.

Miah, M. O. and Kong, J. (2024). Augmented reality and cross-device interaction for seamless integration of physical and digital scientific papers. *International Journal of Human–Computer Interaction*, 41(11):7040–7057.

Montanares Mundaca, D. C. (2019). Desarrollo de un módulo de búsqueda web para el software EDDIE. Trabajo de titulación, Universidad de Santiago de Chile.

Orozco Solano, A. (2017). *Diseño de Sistemas Bioinformáticos y Biología Computacional aplicados en Nanobiotecnología Molecular Médica*. PhD thesis, Universidad Autónoma de Madrid.

Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66.

Pezoa Riquelme, D. E. (2020). Módulo de consistencia en formato físico y digital para sistema de lectura aumentada no obstrusivo. Trabajo de titulación, Universidad de Santiago de Chile.

Salvucci, D. D. and Goldberg, J. H. (2000). Identifying fixations and saccades in eye-tracking protocols. In *Proceedings of the 2000 Symposium on Eye Tracking Research & Applications (ETRA)*, pages 71–78.

Santibáñez Espinoza, F. (2020). Rediseño centrado en el usuario de las interfaces de EDDIE. Trabajo de titulación, Universidad de Santiago de Chile.

Smith, R. (2007). An overview of the Tesseract OCR engine. In *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, volume 2, pages 629–633.

Wander, S. (s.f.). Enlight: Augmented reading. Proyecto de grado MFA, School of Visual Arts. `https://vimeo.com/93202502`.

Wellner, P. (1993). Interacting with paper on the DigitalDesk. *Communications of the ACM*, 36(7):87–96.

ANEXO A. ERRORES DE EDDIE-2023 Y PLANTILLA DE TRAZABILIDAD

La Tabla A.1 reproduce un extracto de los errores registrados por Ibaceta Jaña (2023) que se citan en este documento. La matriz completa se construye con la plantilla de la Tabla A.2, una fila por cada uno de los 67 errores, ejecutando el caso de prueba original en EDDIE-2023 y en la versión en Python.

Tabla A.1: Extracto de errores encontrados en EDDIE-2023.

ID	Tipo	Descripción	Estado
E7	Interno-otro	El programa no se puede inicializar por falta de cámara disponible.	SF
E8	Interno-otro	El programa no funciona correctamente sin dos cámaras disponibles.	SF
E9	Externo	Falta de conexiones eléctricas en el laboratorio.	SP
E10	Externo	Falta de seguridad en el computador del laboratorio.	SF
E11	Interno-consistencia	La sincronización hace fallar el programa cuando no se ingresa un PDF.	SF
E12	Externo	No existe un manual de instalación.	SS
E13	Externo	No existe un manual de uso.	SS
E14	Interno-otro	Los mensajes de error están en un idioma distinto al resto de la aplicación.	SF
E15	Externo	No se reconoce el eye tracker en el computador.	SF
E16	Externo	La conexión de la cámara digital es inestable.	SP
E21	Interno-consistencia	El botón de búsqueda hace fallar el programa si no se sube un PDF.	SF
E22	Interno-consistencia	La revisión del contenido del PDF falla si no existe el archivo.	SF
E27	Interno-otro	Faltan comentarios en el código.	SS
E28	Interno-otro	Variables y funciones sin nombres representativos.	SS
E30	Interno-consistencia	El contenido del PDF falla si no se indica una página.	SF
E48	Interno-consistencia	La consistencia de comentarios falla con datos correctos.	SF
E49	Externo	Con TheEyeTribe no se detecta la parte inferior de la proyección.	SS
E50	Interno-consistencia	No se encuentra la carpeta `PluginsConsistencia`.	SF

Fuente: Ibaceta Jaña (2023). SF: solución final; SP: solución parcial; SS: sin solución.

Tabla A.2: Plantilla de la matriz de trazabilidad.

ID	Módulo	Estado C#	Cat.	¿Reproducido?	Resultado en Python	Evidencia
E*n*	...	SF/SP/SS	(a)–(g)	Sí/No	Corregido / Persiste / Fuera de alcance	Prueba, log o nota de alcance

Fuente: Propuesta de tesis (Tabla 3.3).

ANEXO B. PLAN DE TRABAJO

Tabla B.1: Plan de trabajo.

Semanas	Fase	Actividades	OE	Horas	Entregable
1–3	Plan	Diagnóstico de la arquitectura en C# mediante la reclasificación de errores y prototipos desechables de factibilidad.	OE1	108	Diagnóstico y resultados de prototipos
4–6	Do (diseño)	Diseño del núcleo y de los contratos de interfaz; primer prototipo funcional.	OE2	108	Arquitectura y contratos
7–10	Do (implem.)	Núcleo, capa de abstracción de sensores, consistencia, reconocimiento de texto e interacción gestual.	OE3	144	Prototipos funcionales del núcleo con módulos
11–12	Do (implem.)	Rastreo ocular con hardware físico; contingencia: validación con `EyeTrackerMock`.	OE3	72	Prototipo integrado
13–14	Check	Pruebas de software: cobertura, trazabilidad, latencia y métricas de sincronización y visión.	OE4	72	Reporte de pruebas de software
15–16	Check	MECIVA PS por módulo, validez y confiabilidad, pruebas de aceptación.	OE4	72	Reporte de verificación y validación
17	Act	Ajustes finales, cierre de la documentación y entrega.	OE4	36	Documentación y entrega final

Fuente: Propuesta de tesis (Tabla 6.1).

APÉNDICE A. FICHAS DE LOS AVANCES

Cada avance se describe con el formato de Figueroa Contreras (2025): identificador, nombre, objetivos, descripción y requisitos funcionales y no funcionales. Las tablas se generan desde la carpeta `avances/` del repositorio.

Tabla A.1: Primer prototipo - Núcleo orquestador y contratos de plugins.

ID Prototipo	P-01
Nombre	Núcleo orquestador y contratos de plugins
Objetivos	Definir PluginBase y los contratos de cada módulo (ICameraSensor, IImageProcessor, IGesturePlugin, IEyeTracker, IConsistencyProvider, IWebSearchProvider, ISpeechProvider). Implementar el OrchestratorCore (inicializar $\rightarrow$ ejecutar $\rightarrow$ liberar) y el PluginLoader con un único mecanismo de carga. Validar la inyección de dependencias sensor$\rightarrow$consumidor, el aislamiento de fallos y la latencia por ciclo.
Descripción	Primera versión del núcleo, sin interfaz. Se prueba con dispositivos simulados (MockCameraSensor, EyeTrackerMock) y con plugins de prueba: un consumidor de frames, un consumidor de resultados del ciclo, un plugin que falla cada 10 ciclos y una entrada con una clase inexistente (equivalente a la DLL faltante de los errores E38/E50). El OrchestratorCore de este avance es la versión temprana del repositorio (sin overrides, invoke ni reinicio en caliente).
Requisitos Funcionales	N1, N2, N3, RFCO12
Requisitos No Funcionales	RNF-01, RNF-02, RNF-04, RNF-05

Fuente: Elaboración propia, 2026.

Tabla A.2: Segundo prototipo - Capa de abstracción de hardware y módulo de consistencia.

ID Prototipo	P-02
Nombre	Capa de abstracción de hardware y módulo de consistencia
Objetivos	Incorporar DeviceFactory para conmutar cada dispositivo entre real y simulado sin cambiar código. Implementar la cámara real (OpenCV, hilo de captura) y la enumeración de cámaras de Windows. Integrar primero el módulo sin dependencia de hardware: la consistencia papel↔PDF como plugin IConsistencyProvider (PyMuPDF). Extender el núcleo con overrides de configuración, invocación bajo demanda, reinicio en caliente y traza de ejecución.
Descripción	El núcleo pasa a su versión definitiva y se agrega la capa HAL. La consistencia (resaltados, marcadores, comentarios y figuras, búsqueda de texto) se ejecuta a través del orquestador con OrchestratorCore.invoke(). Sigue sin interfaz: se demuestra con demos/demo_consistencia.py, que sincroniza tres «fotos» sintéticas de la hoja con un PDF de prueba y guarda el PDF resultante y el render de cada página.
Requisitos Funcionales	N4, N5, N6, N8, C1, C2, C3, C7, C9, C10, C11 (parcial)
Requisitos No Funcionales	RNF-01, RNF-02, RNF-04, RNF-05

Fuente: Elaboración propia, 2026.

Tabla A.3: Tercer prototipo - Reconocimiento de texto e interfaz de configuración.

ID Prototipo	P-03
Nombre	Reconocimiento de texto e interfaz de configuración
Objetivos	Integrar el reconocimiento del texto resaltado (IImageProcessor: segmentación del resaltado + Tesseract) sobre la cámara de texto declarada. Construir la ventana de configuración a partir de los controles extraídos de los .Designer.cs, con un diseño propio en español (ui/theme.py, ui/modern_forms.py) y las páginas Texto (OCR) y Documento PDF. Ejecutar el ciclo del orquestador dentro de Qt (EddieRuntime) y persistir los ajustes. Poder probar el OCR y ver el PDF cargado desde estas páginas, sin esperar la pantalla de proyección.
Descripción	Aparece la interfaz. Conserva todos los controles del original (mismo nombre, tipo, rangos y estado inicial), ordenados en tarjetas con layouts de Qt, con un menú lateral que muestra el estado de cada módulo y una pantalla de inicio «Preparar sesión». Solo se construyen las páginas de los módulos ya integrados: las demás aparecen en el menú deshabilitadas con el avance en que se integran, y «Iniciar proyección» avisa que la proyección llega en P-06. Como en el C# el texto reconocido solo se veía desde la proyección, se agregan el botón «Reconocer texto» con el texto reconocido y una vista previa del PDF con navegación de páginas. Los mensajes son avisos no modales.
Requisitos Funcionales	FD1, N7, U1, U3, RFS3, RFS5, RFS6, RFS7, RFS8
Requisitos No Funcionales	RNF-01, RNF-06, RNF-07

Fuente: Elaboración propia, 2026.

Tabla A.4: Cuarto prototipo - Interacción gestual.

ID Prototipo	P-04
Nombre	Interacción gestual
Objetivos	Integrar los plugins gestuales del C# (Color Pen, Hand Skin, Mouse) bajo IGesturePlugin, compartiendo la cámara gestual declarada. Portar la calibración sensor→aplicación, el control del cursor («Recognition on»), la máquina de estados del clic y la captura automática de la selección. Agregar la página Gestos, con calibración asistida («Fijar inicio» / «Fijar fin» desde la lectura del sensor).
Descripción	Los tres detectores heredan una base común (CalibratedGesturePlugin) que aplica la calibración y el clic; cada uno solo implementa su detección. Este avance porta el comportamiento del C# tal cual, incluida la rotación fija de 180° de Hand Skin, que después se detectó como defecto (H-01).
Requisitos Funcionales	G1, G2, G3, U2
Requisitos No Funcionales	RNF-01, RNF-04, RNF-06

Fuente: Elaboración propia, 2026.

Tabla A.5: Quinto prototipo - Rastreo ocular.

ID Prototipo	P-05
Nombre	Rastreo ocular
Objetivos	Unificar GazePoint (Open Gaze API), TheEyeTribe y GazeCloud bajo IEyeTracker, con EyeTrackerMock como contingencia (DeviceFactory). Portar la sesión de mirada: control del mouse por mirada, clic por permanencia, retícula y registro CSV. Agregar la página Rastreo ocular (exportar/importar configuración).
Descripción	Los tres protocolos se prueban con servidores falsos que emiten el mismo formato que los dispositivos reales. La sesión de mirada es un consumidor de resultados del ciclo: recibe la muestra del eye tracker en el mismo ciclo en que se produce.
Requisitos Funcionales	RO1–RO7, RFS4
Requisitos No Funcionales	RNF-01, RNF-02, RNF-05

Fuente: Elaboración propia, 2026.

Tabla A.6: Sexto prototipo - Pantalla de proyección, búsqueda con LLM y voz.

ID Prototipo	P-06
Nombre	Pantalla de proyección, búsqueda con LLM y voz
Objetivos	Integrar la pantalla de proyección (ProjectionScreenActivity2): modo resaltado/captura, búsqueda y sincronización papel↔PDF, con los botones en la franja superior (zona de mayor validez del eye tracker según Ibaceta, 2023) y la zona de lectura y las cruces en su posición original. Responder los cinco servicios de búsqueda con Gemini 2.5 Flash bajo IWebSearchProvider, en lugar de las APIs vencidas (Wikipedia, Google, Bing, YouTube, Cloud Vision). Integrar lectura en voz alta y comandos de voz (ISpeechProvider) y las páginas Interacción y Búsquedas.
Descripción	Con este avance el sistema queda completo y equivale al código de «Proyecto final» antes de la verificación. Es una extensión respecto del alcance del seminario, donde IWebSearchProvider quedaba solo como interfaz.
Requisitos Funcionales	B1–B13, C5, C6, C8, C11, C12, RFS1, RFS2
Requisitos No Funcionales	RNF-01, RNF-06, RNF-08

Fuente: Elaboración propia, 2026.

Tabla A.7: Séptimo prototipo - Versión final verificada.

ID Prototipo	P-07
Nombre	Versión final verificada
Objetivos	Verificar el sistema integrado: cobertura, latencia, trazabilidad de errores y pruebas de regresión. Corregir los hallazgos de la verificación (H-01, H-02, H-03) y dejar una prueba que los detecte. Cerrar la documentación técnica (README, EQUIVALENCIAS, DISENO_INTERFAZ).
Descripción	Es el contenido de «Proyecto final» con las correcciones de la verificación. Cada hallazgo tiene una prueba de regresión que falla con el código de P-06 y pasa con el de P-07.
Requisitos Funcionales	Todos los anteriores
Requisitos No Funcionales	RNF-01, RNF-02, RNF-03, RNF-04, RNF-05, RNF-06, RNF-07, RNF-08

Fuente: Elaboración propia, 2026.

Tabla A.8: Hallazgos de la verificación.

ID	Descripción	Estado	Justificación o Solución
H-01	Al calibrar con Hand Skin la vista de la cámara aparece rotada 180°. HandSkinRecognition.cs rotaba siempre la imagen porque la cámara del laboratorio estaba montada invertida; el port conservó esa rotación fija, solo en ese plug-in (Color Pen y Mouse no rotan), así que la vista previa y las coordenadas de calibración quedaban al revés con cualquier otra cámara.	SF	La rotación pasa a ser una opción común de los tres plug-ins (rotate_180, por defecto desactivada) que se aplica una sola vez al frame; vista previa, coordenadas y recorte comparten orientación. Casilla nueva «Rotar la imagen 180°» en Gestos para el montaje invertido del laboratorio.
H-02	ConsistencyPlugin guardaba cada cambio con saveIncr(). Con PyMuPDF 2.1, desde el segundo guardado incremental el PDF queda con la xref dañada; al reabrirlo PyMuPDF lo «repara» y se pierden los resaltados, comentarios y marcadores guardados antes. En pantalla no se notaba porque el documento en memoria seguía correcto.	SF	Cada guardado escribe una copia completa en un archivo temporal, reemplaza el original y vuelve a abrirlo (se mantiene el respaldo .bac). Si otro programa tiene el PDF bloqueado (Windows), se reintenta y luego se informa con un mensaje claro.
H-03	Al pulsar los botones de carpeta o las casillas de Rastreo ocular aparecía «'bool' object has no attribute 'mapToGlobal'». La ayuda contextual se conectaba con lambda w=widget, t=text a la señal clicked; en los botones, clicked(bool) entrega checked y Qt lo pasaba como primer argumento, reemplazando w por False. La acción del botón sí se ejecutaba, pero cada clic dejaba una excepción.	SF	La ayuda por clic queda solo para los íconos de información; los botones y casillas la muestran al pasar el mouse. La lambda ignora los argumentos de la señal (lambda *_, w=widget, t=text). Se corrige desde P-05, primer avance con esos controles, hasta P-07.

Fuente: Elaboración propia, 2026.

APÉNDICE B. MEDICIONES DE LOS PROTOTIPOS

La Tabla B.1 resume el tipo de medición y el resultado principal de cada prototipo. El detalle (protocolo, fórmula, parámetros y todas las métricas) está en `avances/P-00_prototipos_desechables/evidencias/MEDICIONES.md`, generado junto con los valores de este documento.

Tabla B.1: Resumen de las mediciones de los prototipos.

ID	Tipo	Resultado principal
F1	laboratorio	[Pendiente: FPS y latencia con las cámaras]
F2	sintético	Eficacia Otsu 25,8 %; adaptativo 96,8 %
F3	pdf	CER 1,46 % / WER 6,86 % (sin preproceso: 0,42 % / 0,71 %)
F4	sintético	Éxito 97,5 %, RMSE 2,26 px
F5	sintético	Detección 100 % sin rojo proyectado; 0 % con rojo
F6	sintético	IoU 96,5 %, punta correcta 96,5 %
F7	sintético	Error de reproyección 1,53 $\pm$ 0,51 px
F9	laboratorio	[Pendiente: latencia de eventos]
F10	sintético	Pérdida 0 %, *jitter* 0,45 ms
F11	código	0 fallas en 240 cargas, 0,36 ms
F12	sintético	100 % voluntarias, 3,5 accidentales/min
F13	sintético	0,82 ms por cuadro
F14	pdf	Coherencia 82 %
F15	pdf	48/48 persistidos, 13,8 ms
F16	sintético	100 % detectadas, 0,68 pt
F17	sintético	Exactitud 80 %, IoU 90,1 %
F24	laboratorio	[Pendiente: exactitud de comandos]
F25	laboratorio	[Pendiente: palabras por minuto]
F27	sintético	0,58 ms por cuadro

Fuente: Elaboración propia, `mediciones/medir_prototipos.py` (P-00), 2026-11-08.

Para repetir las mediciones en el PC y agregar las de hardware:

```
cd avances\P-00_prototipos_desechables
python mediciones\medir_prototipos.py --pdf ruta\al\documento.pdf --camara 0
python ..\..\tesis\datos\actualizar_datos.py ..
```

El segundo comando copia los valores nuevos a `datos/`, y al recompilar el documento todas las tablas y fichas se actualizan.

APÉNDICE C. COMPARACIÓN DE LA INTERFAZ

Las figuras siguientes comparan la interfaz de EDDIE-2023, reproducida a partir de sus archivos `.Designer.cs`, con la versión en Python.

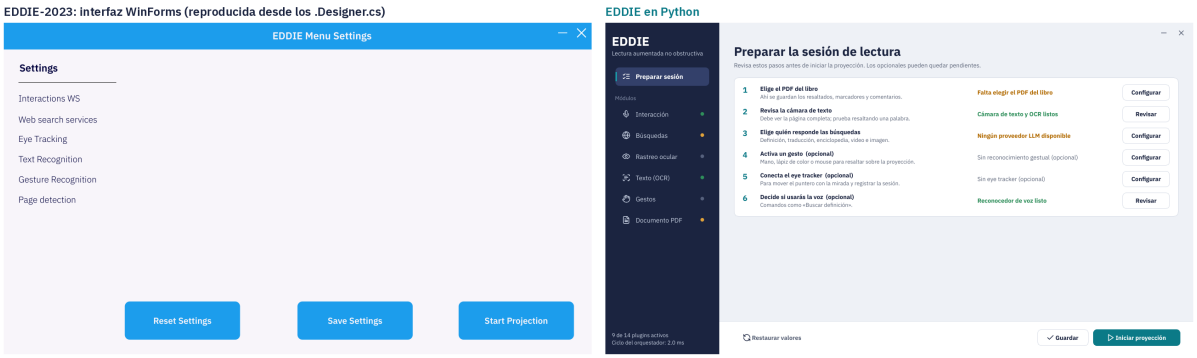


Figura C.1: Pantalla de inicio: EDDIE-2023 (izquierda) y versión en Python (derecha).
Fuente: Elaboración propia.

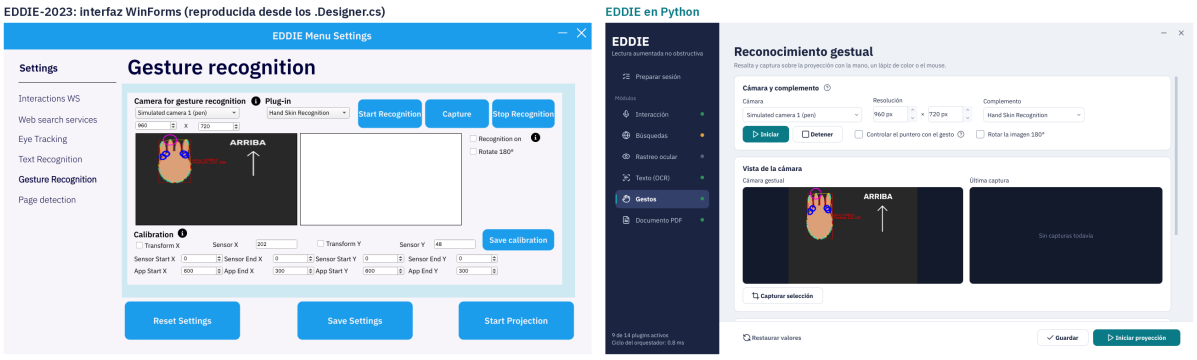


Figura C.2: Configuración de gestos: EDDIE-2023 (izquierda) y versión en Python (derecha).
Fuente: Elaboración propia.

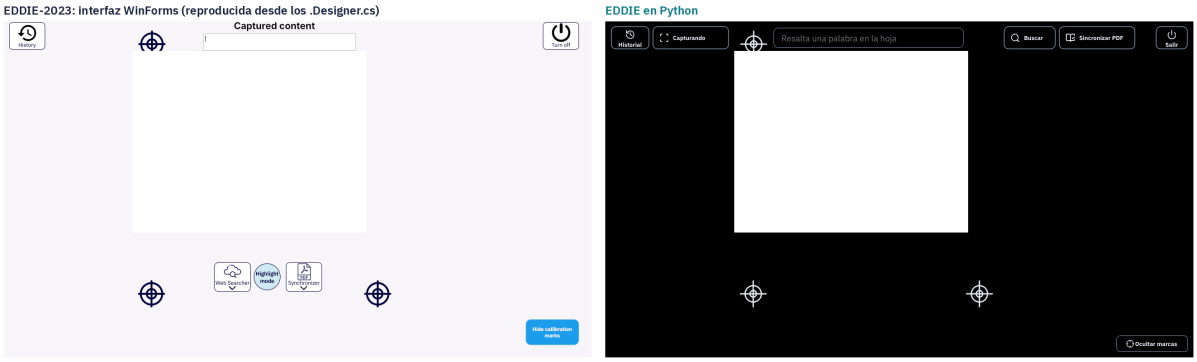


Figura C.3: Pantalla proyectada: EDDIE-2023 (izquierda) y versión en Python (derecha).

Fuente: Elaboración propia.

APÉNDICE D. MANUAL DE INSTALACIÓN Y USO

D.1 INSTALACIÓN

1. Instalar Python 3.12 (64 bits) y, para el reconocimiento de texto, Tesseract-OCR con el idioma español.

2. En la carpeta del proyecto:

   ```
   python -m venv .venv
   .venv\Scripts\activate
   pip install -r requirements.txt
   ```

3. Para las búsquedas, copiar `config/secrets.example.json` como `config/secrets.json` y pegar la clave de Google AI Studio, o definir la variable `GEMINI_API_KEY`. Este archivo no se sube al repositorio.

4. Para el rastreo ocular, abrir GazePoint Control o EyeTribe Server antes de conectar el eye tracker desde la aplicación.

D.2 EJECUCIÓN

```
python main.py              # con el hardware del laboratorio
python main.py --sim        # con cámaras y eye tracker simulados
python -m pytest            # pruebas automatizadas
python tools\generar_evidencias.py   # evidencias
```

D.3 USO

1. Al abrir, la pantalla de inicio muestra los pasos para preparar la sesión y el estado de cada módulo; cada paso lleva a su página.

2. En *Texto* y *Gestos*, elegir la cámara y la resolución. En gestos, elegir el complemento (lápiz, mano o mouse), iniciarlo y calibrar apuntando a la cruz superior izquierda y luego a la inferior derecha.

3. En *Documento PDF*, elegir el PDF del libro.

4. En *Rastreo ocular*, elegir el eye tracker, la retícula y el registro CSV, y pulsar *Aplicar y conectar*.

5. Guardar e iniciar la proyección. En la proyección, resaltar una palabra; el texto reconocido aparece en el campo superior y los botones de búsqueda entregan definición, traducción, enciclopedia, video o imagen. *Sincronizar PDF* abre las herramientas de consistencia papel–PDF.

Los ajustes se guardan en `config/user_settings.json`, las búsquedas en `CSV_registro_actividades/` y cada acción del orquestador con su latencia en `logs/`.